

MINISTÉRIO DA CIÊNCIA E TECNOLOGIA
INSTITUTO NACIONAL DE PESQUISAS ESPACIAIS

INPE

INDUSTRIAL PATTERN SEQUENCING PROBLEMS: SOME COMPLEXITY
RESULTS AND NEW LOCAL SEARCH MODELS

Alexandre Linhares

Tese de Doutorado em Computação Aplicada, orientada pelo Dr. Horacio Hideki Yanasse,
defendida em 17 de Maio de 2001.

INPE
São José dos Campos
2001

Em memória a

Ciro Santos Linhares

(1945-2000)

Index

CHAPTER 1- Introduction	1
1.1. Pattern sequencing problems	1
1.2. Modern heuristic methods	3
1.3. outline of the thesis	4
CHAPTER 2 - Minimization of open stacks: some complexity results	7
2.1. Introduction	7
2.2. Computational complexity of the MOSP.	8
2.3. Related pattern-sequencing problems	15
2.4. A new conjecture	21
2.5. Chapter Summary	23
CHAPTER 3 - Linear Gate Assignment: a preliminary heuristic proposal	25
3.1. Introduction	25
Simulated Annealing and the Microcanonical Approaches	29
3.3 Numerical Results	35
3.4 Chapter summary	43
CHAPTER 4 - Coordination policies for collective search models	46
4.1 Introduction	46
4.2 The use of 'Distance metrics'	47
4.3 distribution Coordination policies	49
4.4 Re-distribution coordination policies	55
4.5 Chapter summary	56
CHAPTER 5 - Distance metrics for sequencing problems	58
5.1 Introduction	58
5.2 Complexity of the 2-opt distance metric	61
5.3 The 2-Exchange Operator	63
5.4 The insertion operator	66
5.5 Distribution of the insertion and the 2-exchange distance metrics	70
5.6 Chapter summary	71
CHAPTER 6: An illustration of the re-distribution coordination policy	73
6.1 Introduction	73
6.2 System architecture	74
6.3 collective dynamics	77
6.4 Interaction effects on the cost of solutions	84
6.5 Comparison with OPTimal results	88
6.5 Conclusion	91
CHAPTER 7: Conclusion	94
7.1. Summary of findings	94
7.2. possible extensions to this research	96
REFERENCES	98

Preface

Here is a scene that marked my professional life: as I was accepted into graduate school many years ago, I told my great undergrad professor, Prof. Bonassis, that I was going for a Master's degree, and he replied with the following phrase:

— cola no cara bom! (Loose meaning: 'stick with the best guys!').

"Cola no cara bom!", he said many times.

This has been my philosophy ever since; and I have never had the chance to thank him for it. It is amazing how such a simple advice can assist you over and over for years. The following story is nothing but a corollary of the repeated application of his advice.

=====

The preface can be one of the hardest parts of a thesis to put down on paper, as I, in a deep sense, feel I owe these pages to many people. I think I will proceed slowly here, going gradually from the formalities to the personal appreciations. As a first note, since part of this work was conducted in Canada and in the United States, and given the standard of communication between scientific communities, I have decided to actually write the thesis on English. I hope that this does not appear in any way to my Brazilian friends as elitist or arrogant, but instead as a deliberate pragmatic decision.

I must acknowledge the financial support of the CAPES foundation, which I received during the first year. I then applied to FAPESP for financial support, and the project was subsequently accepted. Thus, FAPESP grant number 97/12785-8 financed the remaining period, including my international visits. It is great for a country such as Brasil to have an institution of the stature of FAPESP, and I hope someday the state of Rio de Janeiro will be

able to develop FAPERJ into something more like FAPESP. I would also like to thank the Fundação Getulio Vargas, specially Profs. Deborah and Bianor, for believing (and investing) in me, at the time (a lowly form of life:) a Ph.D. in the making, and for their full support of the final months of development of this work.

Some acknowledgements are necessary. In Chapter 2, for instance, it has come to our notice that Prof. Carlo Alberto Bentivoglio has also derived proposition 2.2 independently (MOSP is NP-Hard) (personal communication, May 1999). In Chapter 3, Professor José Carlos Becceneri of the Brazilian Space Research Institute computed the improved lower bounds given on table 3.4. We must thank Professor Sao-Jie Chen, of National Taiwan University, for providing us the gate matrix layout circuits, and for pointing out additional references. We are also grateful to Professor John R. Ray, of Clemson University, for calling our attention to other approaches to simulated annealing in the microcanonical ensemble. Thanks are also due to Dr. Tetsuo Asano, of JAIST - Japan Advanced Institute of Science and Technology -, and to Dr. Chung-Kuan Cheng, of University of California at San Diego. Chapter 4 was conducted mostly in the *center de recherche sur les transports*, and thus we thank Prof. Gilbert Laporte for his kind invitation. In Chapter 5, Maria Cristina Nogueira Gramani participated in the proof of correctness of the algorithm concerning the insertion distance metric. In Chapter 6, we thank Prof. Bentivoglio for providing us the MOSP problem instances; we must also thank Prof. Wascher and Profs. Fink and Voss for providing us their problem data, even though they were not used for technical reasons.

Many researchers, such as Alain Hertz, and Kathryn Stecké, were also contacted concerning problem data for the MTSP. Unfortunately, we have not been able to test the current ideas on this problem, but this is definitely one future extension to this thesis.

I would also like to thank those researchers contacted for a possible research visit: Dr. Alan Zinober of the University of Sheffield, Dr. Katherine Dowsland of The University of Swansea, Dr. John Beasley of Imperial College.

=====

The story tells us that Sisyphus carries a huge rock up some interminably long stairs; and, as he approaches the end of the stairs, he lets the rock slip from his hands, and it rolls down all the way, forcing him to go down and start all over again; which he does without a second thought. It is said that this process goes on throughout the ages. Sometimes it is indeed difficult to start over. Over the course of this thesis I have lived in five different cities in three different countries. In each city I found a new laboratory, new ‘work’ friends, new ‘squash’ friends, new *real* friends, and maybe even a new emphasis on research. It has been an awesome experience.

The first two years were spent at São José dos Campos, where I have made some very good friends, such as the never-quiet Marcelo and all “the guys at the lab”, such as Alvaro, Nelson, Jean, Geraldo, and later Eduardo, Maju, Daniel.... (this list goes on and on). At the lab we had good discussions about classical operational research problems, and of course the MOSP. At this time Patty was a great source of support. I would like to thank my out-of-INPE friends in SJC: Diogo, Rodrigo, Tardelli, and specially Gil.

I then moved to Campinas, where I stayed for a while on the DENSIS lab at UNICAMP. The emphasis on the discussions there changed to evolutionary computation, mostly with their passionate supporters Alexandre and Pablo, and about classical operations research with the remaining group at the lab. I also had some good friends from outside the university, especially Fabio, and Rodrigo. Of course, there was also Cris, who shared many dreams (and good laughs) with me.

The third city in a row was beautiful Montreal, where I worked in the *Centre de recherche sur les transports*, a famous lab specialized in transportation, but with broader interests in optimization problems. I must thank Prof. Gilbert Laporte for his very kind invitation and warm reception.

Some months later, I crossed the border to the Unites States, and drove across ten states, in one of the most beautiful trips I ever made, to land on Bloomington, Indiana, where I worked with Douglas Hofstadter and Harry Foundalis on cognitive science. This was a turning point for me, and I had the opportunity to play around with some wonderful topics in artificial intelligence, especially with the shaky philosophical foundations of the field. I would very much like to thank the Intelligence Theorist Douglas Hofstadter, as his Pulitzer prize winning

work served as the very foundation of my views concerning the possibility of artificial intelligence. In fact, my first book review on the subject concerned his wonderful *Fluid Concepts and Creative Analogies*, and my first paper in the field concerned the subject of one of his long-term scientific goals, the solution of the incredibly challenging *Bongard Problems*. I very much appreciated his invitation to visit the Center for Research on Concepts and Cognition at Indiana University at Bloomington, and the very warm reception that I had there. I also very much appreciated the invitation from Harry Foundalis to share a great *campus view* apartment. Harry, by the way, is a guy with the perfect Ph.D. thesis topic, and he became in time a great friend of mine. I am sure Harry's thesis is going to shake up the world whenever it comes up (and I hope that our joint upcoming paper profoundly disturbs as many people as possible ☺). In Bloomington I also had the opportunity to meet another incredible cognitive scientist, Brian Cantwell Smith. I cannot forget to mention my good Bloomington friends Biro, Bill Ziegler, and the guys at the CRCC, such as John, Hamid, Ms. Helga Keller, and, of course, specially, Mehriban, my running-away-from-the-police-and-the-FBI partner ☺. I am very thankful to all these people for the great time I have had in Bloomington.

Finally, the fifth city is my own hometown, the city I originally left to study for the doctorate: Rio de Janeiro. Here I also had a warm, not to mention lucky, reception: in ten days of arrival, I was offered a great job at the business school of FGV. In Rio I have once again the support of Prof. Torreão, and the faculty of the Institute of Computing at UFF. On the personal side, I have had some good laughs with Mariana; and have had the opportunity to meet good old friends, such as (the upcoming millionaire) Sebastian, and Fernando, who joins me (after IBM) for a second time as a colleague at work. These people made the 'final days of writing' seem better than days of sheer agony.

Some people were always there for me, even when I did not deserve it fully. My good friend Dinho (and now, skydiving partner), my Brother Rick (who is nowadays inseparable from Flavia ☺), and of course my Mother, who has displayed great amounts of strength these last months. These are the people that make it all worth it.

My thesis advisor, Dr. Horacio Hideki Yanasse, has been a good friend over these years. Always a rigorous academic, always friendly, and always in a good mood, he even managed

to be, sometimes, available for discussion! ☺ His ability to *just spot* the weak link in a line of reasoning has always impressed me; it has improved my thesis in countless points.



The thesis defense: From the left, the comitee is composed by Dr. Horacio Hideki Yanasse, Dr. Luis Antonio Nogueira Lorena (INPE), Dr Ney Soma (ITA), Dr. José Ricardo A. Torreão (UFF), and Dr. Nelson Maculan (UFRJ)

=====

Looking back on time, probably the one individual who mostly contributed to my intellectual advancement – and to my views of the world – was my cousin Antonio De Bellis. I was deeply shattered by his premature death. He was the kind of person that would always bring up a deep insight to any conversation, even the most ordinary ones. If you met him for just five minutes, the silly jokes and talk of this ordinary encounter could stay in your mind for days. Antonio was so creative and insightful that he just could not be superficial. He

instilled in me the interest for many fields, such as metaphysics (“Does the absolute exist?”), he would ask – and then engage you in an amazingly extraordinary conversation), ethology, and cognitive science. It is a true tragedy – not only for me, and not only for his family — that we have lost him.

After his death, I posed myself the following question: was it worth it? All those years of independent study, immersed in deep abstract questions about the nature of the world, of god, of life, and of mind, were them of any value at all? While at first I thought that all that was really thrown away after his death, after a second thought I could finally come to see the true answer. Yes, of course it was worth it, for that was one of the things that made him special in the first place, and that made his life incredibly extraordinary. It was probably after this very thought that I finally embarked on graduate school with such enthusiasm; had I not realized this at that time, I would probably never engage myself to a Ph.D. degree. His exalted enthusiasm contaminated me; and there is definitely a part of him in these pages.

=====

It is amazing how we cannot explain the things that go on in our minds; the visions we have for the future. After I started the doctorate, I sometimes, as any ordinary Ph.D. student, visualized the glorious day when I would be defending my thesis in front of an eminent committee. There I was, full of enthusiasm, performing my presentation, with a full room paying close attention. I don’t know why, but every time that I had this thought, I could see one particular person quietly sitting there to support me; no other person appeared as much as this particular one. It was my father. I simply have no idea why I always visualized him in the defense. Maybe it is because his presence always gave me immense enthusiasm. It is deeply painful to realize that he could not be there, for the thesis, and for the rest of my life. It was arduous to find enthusiasm to work without his encouragement.

The poet Robert Bly, in his beautiful, mythological, book, *Iron John*, writes about the figure of the father, in terms of a king. The following passage captures, at least to me, what was the very essence, the very spirit of my father. Those who knew him will instantly recognize him in the passage:

“The King in his upper room comes towards us with a shining face – he blesses, he encourages creativity, he establishes by presence alone an ordered universe.”

Robert Bly, *Iron John*, 1990

I thank you for the blessings, daddy; and I thank you for all the encouragement; and I thank you for your shining face.

It has been more than a year without your presence;
and there is still much disorder in the universe.

Alexandre Linhares

INDUSTRIAL PATTERN SEQUENCING PROBLEMS: SOME COMPLEXITY RESULTS AND NEW LOCAL SEARCH MODELS

ABSTRACT

In this thesis we explore some industrial pattern sequencing problems arising in settings as distinct as the scheduling of flexible machines, the design of VLSI circuits, and the sequencing of cutting patterns. This latter setting presents us the *minimization of open stacks problem*, which is the main focus of our study. Some complexity results are presented for these sequencing problems, establishing a surprising connection between previously unrelated fields. New local search methods are also presented to deal with these problems, and their effectiveness is evaluated by comparisons with results previously obtained in the literature. The first method is derived from the simulated annealing algorithm, bringing new ideas from statistical physics. The second method advances these ideas, by proposing a collective search model based on two themes: (i) to explore the search space while simultaneously exhibiting search intensity and search diversity, and (ii) to explore the search space in proportion to the perceived quality of each region. Some preliminaries, given by coordination policies (to guide the search processes) and distance metrics, are introduced to support the model.

**INDUSTRIAL PATTERN SEQUENCING PROBLEMS: SOME COMPLEXITY
RESULTS AND NEW LOCAL SEARCH MODELS**

**(PROBLEMAS INDUSTRIAIS DE SEQUENCIAMENTO DE PADRÕES: ALGUNS
RESULTADOS DE COMPLEXIDADE E NOVOS MODELOS DE BUSCA LOCAL)**

RESUMO

Nesta tese nós exploramos alguns problemas industriais originários de tarefas tão distintas quanto a programação de uma máquina flexível, o projeto de circuitos integrados VLSI e o sequenciamento de padrões de corte. Desta última tarefa provêm o problema de minimização de pilhas em aberto que é o principal foco do estudo. Alguns resultados de complexidade são apresentados para estes problemas estabelecendo-se uma conexão entre áreas que previamente não pareciam estar relacionadas. Novos modelos de busca local também são apresentados para lidar com estes problemas e sua eficácia é medida por comparações com resultados previamente estabelecidos na literatura. O primeiro método é derivado do algoritmo de *simulated annealing* trazendo conceitos da física estatística. O segundo método expande estas idéias, propondo um modelo coletivo baseado em dois princípios: (i) a exploração do espaço de soluções exibindo uma busca simultaneamente intensiva e distribuída e, (ii) a concentração da busca em proporção com a qualidade percebida em cada região. São também introduzidos novos métodos essenciais para dar suporte ao modelo, como políticas de coordenação (dos processos de busca) e métricas de distâncias.

CHAPTER 1

INTRODUCTION

1.1. PATTERN SEQUENCING PROBLEMS

In operations research one deals with the application of mathematical and scientific methods to real, industrial, problems. One specific area of operations research with countless applications in industry is known as combinatorial optimization, that is, the optimization of discrete arrangements with a great number of possibilities. The problems with which we will be concerned in this thesis have roots in industrial applications arising in areas as diverse as cutting stock settings, flexible manufacturing systems, and the design of very-large-scale-of-integration (VLSI) circuits [Yanasse, 1997; Linhares and Yanasse, 2001].

One could classify the problems treated here as *pattern sequencing problems* [Fink and Voss, 1999]. In these problems, there is a number of objects that must be sequenced. The objects can be, for instance, a client order to be produced in a flexible manufacturing system, or a wire in a VLSI circuit, or maybe a cutting pattern, i.e., a specific way to cut a large board (of wood, or glass etc.). A particular sequence, which will minimize the production costs, is desired. They are *pattern-sequencing* problems because each of the objects to be sequenced holds a specific pattern of connections to the remaining objects. For example, in the production of a VLSI circuit, each wire needs to be connected to a number of other wires.

The first industrial problem we will consider arises on cutting stock settings: consider that the cutting patterns have already been generated, and they must be sequenced. Each pattern is composed of smaller piece types, and, as the patterns are cut, the pieces of the same type

are stacked together. However, space around the saw is limited and hence, the number of distinct stacks must be small. The following policy is assumed: a stack is *open* as soon as a new piece type is cut and it remains open until all the pieces corresponding to that stack are cut; only then can the stack be removed. This policy should minimize handling and transportations costs; it should also minimize risks for fragile material, such as glass. Thus, it is desired to minimize the maximum number of open stacks during the whole production run – we will be referring to this problem as the *minimization of open stacks problem*, or MOSP, for short.

In these industrial settings, other possible objective functions could be, for instance, to minimize the maximum time that a stack remains open, a problem we refer to as the *minimization of order spread problem* (MORP), or, still, to minimize the number of production discontinuities, where discontinuities occur whenever a piece type that is cut in a particular instant is not cut in the following instant. This problem is referred to as the *minimization of discontinuities problem* (MDP).

If the number of stacks grows too large, the only remaining policy would be to, given a maximum threshold of stacks, remove the stacks as they exceed this number and bring them back as they are needed while trying to minimize these stack switches. A problem with these very same dynamics arises in flexible machine production systems. Consider a flexible machine that is capable of producing many different items. Each item requires the machine to use a particular combination of tools. However, there is a restriction on the total number of tools that can be held by the machine and thus, some tools must be switched between the production of two different items. It is desired to find a sequence of production of the items that will minimize the number of tool switches. We will refer to this problem as the *minimization of tool switches problem*, MTSP.

A problem that, at least on the surface, does not seem to be similar but, as we will see in chapter 2, is deeply related to the MOSP is named as the *linear gate assignment problem*. This problem arises on some hardware architectures of very large scale of integration (VLSI) circuits such as *gate matrix layout* or *programmable logic array folding*. In these architectures there are vertical wires, called *gates*, and horizontal wires, called *nets*, that interconnect the gates. The objective is to obtain a circuit layout of minimum size. This can be done by putting some non-overlapping nets in the same track, that is, the horizontal space usually occupied by a single wire. The computational problem thus consists in finding a

sequence of gates that minimizes the layout area by maximizing the density of the nets placed in the same circuit row. These problems will be formally defined in the next chapter, and, as we will see, these computational problems are considered intractable. It is widely believed – though not proved – that there is no efficient worst-case polynomial-time algorithm for their solution, and the best that one can expect in the case of large industrial-sized problems is to obtain high-quality solutions (which are not guaranteed to be optimal).

Thus, a part of this thesis is concerned with computational problems *per se*, and is focused on understanding the complexity of such problems and their interrelationships. Another part of the thesis is concerned specifically with methods for solving these problems where some modern heuristics are proposed and the focus is then on studying the dynamics of these heuristics, and the quality of the solutions obtained by them.

1.2. MODERN METAHEURISTIC METHODS

In chapters 3, 4 and 6 we will discuss some modern metaheuristic solution methods. These methods are based on an underlying local search procedure; that is, potential solutions are gradually re-arranged in search of an optimal one. These ‘modern metaheuristics’ differ from classical heuristics in the sense that they are able to avoid being stuck at locally-optimum solutions, as they can continue the search after achieving such points.

Many modern metaheuristics have gained widespread use in the last two decades. Genetic and evolutionary algorithms, probably the most disseminated, employ strategies guided by the process of natural selection, where solutions are considered as individuals competing in phenotype space, and the objective function is seen as a fitness measure of the adaptability of the individual. Thus, individuals spread their genes (constituting parts of a solution) to new generations, and, since the number of genes spread tends to be proportional to the fitness of the parents, the new generations tend to evolve from random solutions to high-quality solutions¹.

¹ It remains in dispute the extent on which these algorithms mirror the course of evolution, as there are fundamental dissimilarities between the natural evolutionary process and the computer models.

Simulated annealing has also been inspired by natural phenomena, more specifically, by analogy to the field of statistical mechanics. Solutions to the problem are modeled as states of a physical system, and the transition probabilities between solutions can then be calculated promptly (according to known physical laws) as a simulated process of annealing gradually converges. As we will see in chapter 3, many variants of this method (including our proposal) have recently appeared.

Another prominent method is known as tabu search, which uses memory structures to guide the search towards promising (and hopefully unexplored) regions of the search space. There are basically two types of memory structure: long term memory and short term memory. The latter are used mainly for preventing the system to revisit previous points of the search, while the former, considerably more sophisticated, enables the system to either intensify the search to a promising region, or diversify the search to a whole new unexplored region.

We must point out clearly what these methods do, as much as what they are not capable of doing. These methods are a practical approach to a great number of combinatorial optimization problems, obtaining high-quality solutions for problems of industrial size in a reasonable timescale. On the other hand, they do not provide any guarantee of optimality (or even of approximation) besides an obvious deviation from lower bounds. In this sense, they must be distinguished from *approximation algorithms* that do in fact provide absolute or relative approximation guarantees. Also, they should not be used for some particular instances with special mathematical properties, as it may be possible to solve these instances exactly [Yanasse, 1996]; and it is also possible to solve exactly the general case for moderately sized problems [Becceneri, 1999]. While these methods are not a *panacea*, they are still an increasingly valuable approach to problems that seem to be intractable.

1.3. OUTLINE OF THE THESIS

It is worthwhile to give a brief outline of the thesis. In Chapter 2 the theoretical questions concerning the computational complexity of the MOSP are considered. It is demonstrated that MOSP belongs to the class of NP-Hard problems, and this fact strongly suggests that there is no efficient algorithm for its solution. Hence, if the conjecture $P \neq NP$ turns out to be

true, as it is widely believed, then the best one can obtain for problems such as MOSP is either an exact solution for a moderately sized problem or a heuristic solution for problems of industrial size. This is the reason for our further proposals, in chapters 3, 4 and 6, of heuristic methods to solve the problem. Other open conjectures pertaining to the relation between the MOSP and other pattern sequencing problems, such as the MDP, the MTSP, or the MORP are also clarified in this chapter.

In Chapter 3 a preliminary heuristic proposal for the solution of linear gate assignment problems that, under the classical formulations, are strictly equivalent to the MOSP (as we will see in chapter 2) is provided. This algorithm is based on local search, and is derived from the well-known algorithm of *simulated annealing*. The algorithm was tested on some benchmark circuits, and our computational results were compared against those of previous papers. The comparisons are favorable to our method, indicating that the method is able to find high-quality solutions in a reasonably small amount of time. Some solutions obtained improve, on the previous best found (on the literature), by as much as 7 tracks; as we will see, this is a significant cost improvement.

Unfortunately, the proposed method lacks robustness; as it is necessary to employ multiple starts to actually obtain such high-quality solutions. In order to improve this situation, we propose in Chapter 4 some coordination policies, i.e., guiding principles for coordinating multiple search processes, envisioning a new method that simultaneously displays a highly distributed search over the search space, and also explores intensively each of these multiple areas.

This method relies on some distance metric algorithms that were not available on the previous literature, and so we develop in chapter 5, the desired algorithms for computing such distance metrics. It is also shown in the chapter that these distance metrics can be used in other contexts, for example, in the studies of landscape analysis that have been receiving increasing attention over the last years; and in the algorithms of computational molecular biology that compare genomic sequences.

In Chapter 6 a new collective model for local search is illustrated. The method clearly demonstrates that what was once seen as a dichotomy, the so-called ‘balance between search intensity and search diversity’, is really not a question of a mutually exclusive choice. In our method, there is search intensity and search diversity; and both occur simultaneously. There

is also an increased search effort to the areas of higher perceived quality. The computational results indicate that there is a significant advantage when the coordination policies are used. Other comparisons with the results of Faggioli and Bentivoglio [1998] show that the method regularly finds optimal solutions.

Finally, in the conclusion, a summary of findings, some final comments over these findings, and some proposals for future research are presented.

CHAPTER 2

MINIMIZATION OF OPEN STACKS: SOME COMPLEXITY RESULTS

2.1. INTRODUCTION

In this chapter the MOSP is considered under a theoretical viewpoint. We consider some open conjectures concerning the computational complexity of the MOSP, and also the relation of the MOSP to similar problems, such as the MTSP or the MDP. The question of whether the MOSP belongs to the class of NP-Hard problems is of great importance, as it indicates whether efficient algorithms for solving it are likely to exist.

Some preliminaries are necessary: Consider a production setting where J distinct patterns need to be cut. Each one of these patterns may contain a combination of at most I piece types. We can define the piece-pattern relationship by a $I \times J$ binary matrix $P=\{p_{ij}\}$, with $p_{ij}=1$ if pattern j contains piece type i , and $p_{ij}=0$ otherwise.

When a pattern is cut, the pieces are stored on stacks that remain fixed around the cutting saw until each stack is completed. Each stack holds only pieces of the same type and it remains open until the last pattern containing that piece type is cut. Difficulties in handling a larger number of distinct open stacks appear, for instance, if the number increases beyond the system capability, some of the stacks must be temporarily removed yielding additional costs, higher production time and higher associated risks (as observed in glass-cutting settings). We are thus interested in the sequencing of the patterns to minimize the maximum number of open stacks during the cutting process (An analogous problem arises on the sequencing of

tasks in a flexible machine with tooling constraints [Yanasse,1997]). An open stacks versus cutting instants matrix $Q^\pi = \{ q_{ij}^p \}$ is defined by:

$$q_{ij}^p = \begin{cases} 1, & \text{if } \exists x, \exists y \mid p^{-1}(x) \leq j \leq p^{-1}(y), \text{ and } p_{ix} = p_{iy} = 1, x, y, j \in \{1, \dots, J\}, i \in \{1, \dots, I\} \\ 0, & \text{otherwise} \end{cases} \quad (1)$$

where π denotes a permutation of the $\{1, 2, \dots, J\}$ numbers, and defining a sequence on which the patterns are cut, such that $\pi^{-1}(i)$ is the order (instant) in which the i -th pattern is cut. Note that matrix Q^π holds the consecutive-ones property for columns [Golumbic, 1980] under permutation π : in each row, any zero between two ones will be changed to one. This is called a *fill-in*. Since we are interested in minimizing the number of simultaneous open stacks, we define the following cost functional:

$$Z_{MOSP}^p(P) = \max_{j \in \{1, \dots, J\}} \sum_{i=1}^I q_{ij}^p \quad (2)$$

and define MOSP as the problem of $\min_{p \in \Gamma} Z_{MOSP}^p(P)$, where Γ is the set of all possible permutations π .

In Yanasse [1997], a nontrivial mathematical formulation is presented, a branch and bound scheme to solve it is proposed, and a number of conjectures are raised. In this chapter we will deal with those conjectures.

The first conjecture was about the computational complexity of the MOSP: is MOSP NP-Hard?

2.2. COMPUTATIONAL COMPLEXITY OF THE MOSP.

Proposition 2.1. *MOSP is NP-Hard.*

Proof. Reduction of Modified Cutwidth to MOSP.

MODIFIED CUTWIDTH (MCUT) [Downey and Fellows, 1995]

INSTANCE: Graph $G=(V,E)$, positive integer K .

QUESTION: Is there a one-to-one function $\pi: V \rightarrow \{1,2,\dots,|V|\}$ such that for all i , $1 \leq i \leq |V|$,

$$|\{ (u,v) \in E : \pi^{-1}(u) < i < \pi^{-1}(v) \}| \leq K \quad ?$$

This problem asks for a linear ordering of the vertices of G on a straight line such that no orthogonal plane that intersects the line will cross more than K edges of G . The optimization version of MCUT asks to minimize K over all possible linear orderings of the vertices of G , i.e., $\min_{p \in \Gamma} Z_{MCUT}^p(G)$, where $Z_{MCUT}^p(G) = \max_{i \in (1,|V|)} |\{(u,v) \in E : p^{-1}(u) < i < p^{-1}(v)\}|$.

We will show how to solve an instance $G=(V,E)$ of MCUT with an algorithm for MOSP. First, construct a corresponding $I \times J$ matrix P where $I=|E|$ and $J=|V|$ where we associate each edge in E to a row number and each vertex of V to a column number. Consider an edge $(u,v) \in E$, $u \in V$, $v \in V$. Let i be the row number assigned to this edge and let j and k be the number of the columns assigned to vertices u and v respectively. Make $p_{ij}=p_{ik}=1$, and $p_{il}=0$, for all $l \neq j$ and $l \neq k$.

Lemma 2.1. *Each edge crossing in a linear layout of the vertices of G corresponds to one and only one “fill-in” to problem matrix Q^π , associated to a permutation of the columns of P with the consecutive-ones property for columns.*

Proof. Consider an ordering π of the vertices of G and an edge (u,v) crossing an orthogonal plane at vertex i . Let x be the row in matrix P associated with this edge. From the construction of P , we have $p_{x,p^{-1}(u)} = p_{x,p^{-1}(v)} = 1$, and $p_{x,y}=0$ for all $y < p^{-1}(u)$ and for all $y > p^{-1}(v)$. At this row, following the column sequence that corresponds to the vertex ordering, there must be a fill-in at position (row x , column i), if $p^{-1}(u) \leq i \leq p^{-1}(v)$, since the associated matrix Q^π holds the consecutive-ones property for columns under permutation π .

Now that this basic relation is established, all that must be done is to ensure that the maximum column sum of the MOSP reflects only the fill-ins. In order to achieve this, we create a matrix P' as follows:

$$p'_{ij} = p_{ij}, \quad \forall i \in \{1, 2, \dots, I\}, \forall j \in \{1, 2, \dots, J\}.$$

Let $C = \max_j \sum_{i=1}^I p_{ij}$, and compute a gap $g = C.J - \sum_{j=1}^J \sum_{i=1}^I p_{ij}$. The rows $I+1, I+2, \dots, I+g$ of P' have only a single nonzero element equal to 1. They are distributed over the columns in rows $I+1, I+2, \dots, I+g$ in such a way that there are exactly $C - \sum_{i=1}^I p_{ij}$ ones on each column so that columns of P' have column sum equal to C . Now, for the corresponding MOSP instance P' [of dimension $(I+g)J$], we have:

Lemma 2.2. $Z_{MOSP}^P(P') = Z_{MCUT}^P(G) + C$.

Proof. Observe that each column of P' has the same number of 1's. Thus, when solving MOSP what is being minimized is the maximum column sum of fill-ins. By lemma 2.1, the solution to this instance of MOSP solves the corresponding instance of MCUT.

Proposition 2.1 closes the conjecture posed earlier [Yanasse 1997; Fink and Voss, 1999].

2.2.1 Relations to VLSI design

One basic characteristic of the MOSP is the consecutive-ones property: a stack is open at the moment that the first piece of each type is cut, and only closed when the last piece of that type is cut. Many combinatorial optimization problems also exhibit the consecutive-ones property. A problem that is intrinsically related to the MOSP arises on VLSI design, and is known as the *gate matrix layout problem* (GMLP).

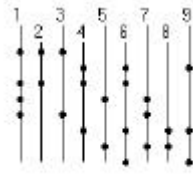


Figure 2.1. A set of gates and transistors of a gate matrix layout style circuit.

A gate matrix layout circuit consists of a set of gates (vertical wires) with transistors (dots) that are used to interconnect the gates. In figure 2.1 such a set of gates before their interconnection is shown. There must be horizontal wires known as *nets* interconnecting all the gates that share transistors at the same position. If we model the problem as a matrix problem, we are given a binary $I \times J$ matrix $P = \{p_{ij}\}$, with $p_{ij}=1$ if gate j holds a transistor at position i , and $p_{ij}=0$ otherwise, then, the implementation of the nets leads to the consecutive-ones property. The wiring between gates is represented by the ones interconnecting the leftmost and rightmost gates, hence, the resultant matrix holds the consecutive-ones property (therefore, we can use the associated matrix Q^π defined previously).

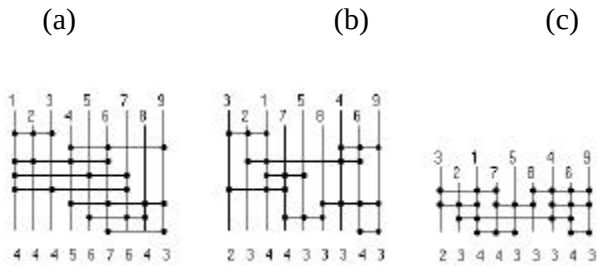


Figure 2.2. Illustration of the *gate matrix layout problem* (GMLP).

One basic feature of the problem is that the sequencing of the gates does not alter the underlying logic equation implemented by the circuit. Furthermore, it is essential to sequence the gates in order to minimize the number of tracks, i.e., the number of necessary physical rows to implement the circuit, given by:

$$Z_{GMLP}^p(P) = \max_{j \in \{1, \dots, J\}} \sum_{i=1}^I q_{ij}^p. \quad (3)$$

Therefore GMLP is the problem of $\min_{p \in \Gamma} Z_{GMLP}^p(P)$.

The number of tracks is the only variable determining the overall circuit area, since the number of gates is constant. In figure 2.2 the gate numbering and number of transistors (plus interconnections) are shown above and below the gates, respectively. In figs. 2.2(a) and 2.2(b) distinct gate permutations are displayed, with the corresponding variation in each column sum. Since the permutation displayed in fig 2(b) yields a maximum column sum of 4, it is possible to pack the layout in 4 tracks (physical rows), as shown in Fig 2(c).

We thus have

Proposition 2.2. *For any matrix P , the number of open stacks $Z_{MOSP}^p(P)$ equals the number of tracks $Z_{GMLP}^p(P)$.*

Proof. Follows trivially from their definitions.

By proposition 2.2 we have that the computation of MOSP and GMLP is equivalent. Using proposition 2.2 we can also prove the NP-Hardness of MOSP, since GMLP is NP-Hard [Kashiwabara and Fujisawa, 1979; Möhring, 1990; Ohtsuki et al., 1979; Wing et al., 1985].

A skeptical reader may argue that “there is no need for proposition 2.1, as proposition 2.2 alone would do the job of proving NP-Hardness of MOSP”. That is true. However, there is an additional issue here. We are not only interested in the theoretical issues of computational complexity, but also in the practical issues of comprehending what kind of problem can actually be solved by an exact MOSP algorithm (such as that proposed in Yanasse [1997]). Consider the following facts: The NP-Hardness of GMLP has been proved by a reduction from the INTERVAL GRAPH AUGMENTATION PROBLEM (IGAP) [Kashiwabara and Fujisawa, 1979]. However, in practice, an exact algorithm for the GMLP (and hence MOSP) cannot be of use to solve IGAP because the number of columns of the GMLP matrix grows very rapidly in relation to the size of the IGAP graph -- the growth is in fact $O(n^2)$ because each column represents an *edge* of the graph [Möhring, 1990]. On the other hand, in the

transformation we have suggested, the number of columns grows only linearly on the MODIFIED CUTWIDTH PROBLEM, because each column represents a *vertex* of the graph. Since the number of possible solutions depends crucially on the number of columns (which are the objects that will be permuted), MODIFIED CUTWIDTH should therefore be more amenable to solution by a MOSP algorithm than INTERVAL GRAPH AUGMENTATION. This is our motivation for developing proposition 2.1.

TABLE 2.1. EQUIVALENT PROBLEMS.

<i>Problem</i>	<i>Discipline</i>	<i>References</i>
MOSP	Operations research	[Yanasse, 1997; Fink and Voss, 1999]
Gate matrix layout	VLSI design	[Möhring, 1990; Wing et al., 1985]
One-dimensional logic	VLSI design	[Ohtsuki et al., 1979]
PLA folding	VLSI design	[Möhring, 1990]
Interval thickness	Graph theory	[Kashiwabara and Fujisawa, 1979]
Node search game	Graph theory	[Kirousis and Papadimitriou, 1985]
Edge search game	Graph theory	[Kirousis and Papadimitriou, 1986]
Narrowness	Graph theory	[Kornai and Tuza, 1992]
Split bandwidth	Graph theory	[Fomin, 1998]
Graph path-width	Graph theory	[Kinnersley, 1992]
Edge separation	Graph theory	[Lengauer, 1981]
Vertex separation	Graph theory	[Kinnersley, 1992]

We must also point out that there is a family of problems related to MOSP that have been studied independently in the literature. In table 1 we have collected a set of problems that consist of, given input F , compute a function $f(F)$ that is either equal to the number of open stacks or closely related to it (plus or minus one).

2.2.2 Fixed parameter tractability

Gate matrix layout is an important problem in the theory of NP-Completeness for the remarkable nonconstructive results obtained recently proving the existence of a polynomial-time decision algorithm for any fixed number k of tracks [Fellows and Langston, 1987]. However, the nonconstructive existence proof provided does not yield the decision algorithm (or any method to achieve it). What this result basically shows is that, for the decision problem where one is given a binary matrix M and a parameter k and asked whether M has a gate matrix layout with k or fewer tracks, the parameter k constitutes an important part of the computational complexity of the problem. If the parameter k is held fixed, the problem becomes theoretically tractable (but not much is changed in practical terms since the fixed parameter algorithm is unknown). These intriguing results have led to the recently suggested *theory of fixed parameter tractability* (FPT) [Downey and Fellows, 1995] that states, for instance,

Theorem 2.1. [Downey and Fellows, 1995; Fellows and Langston, 1987] *GMLP $\hat{=}$ FPT, that is, gate matrix layout is fixed-parameter tractable.*

This result obviously implies that

Corollary 2.1. *MOSP $\hat{=}$ FPT, that is, the minimization of open stacks problem is fixed-parameter tractable.*

If we fix the number of open stacks then there is a polynomial-time algorithm for the decision version of MOSP. Once again, this is mainly a theoretical result since the decision algorithms for each distinct k are unknown. We will return to the issue of fixed parameter tractability in section 2.4.

2.3. RELATED PATTERN-SEQUENCING PROBLEMS

Other related problems studied by Yanasse [1997] include the minimization of tool switches problem (MTSP), the minimization of order spread problem (MORP), and the minimization of discontinuities problem (MDP). These problems, though closely related, are not equivalent to each other or to MOSP (and GMLP).

Let us consider the related problem where there is a maximum capacity of stacks that may be placed around the saw. If a certain cutting sequence exceeds this limit capacity at some point, then one or more stacks will need to be moved, to clear some space for the subsequent processing. These stacks must then be brought back for further processing; an operation that increases the cost and the associated risks of the cutting process. Thus, it is important to minimize the number of stack switches. The minimization of stack switches has been studied in the literature of flexible machines, under the guise of *minimization of tool switches problem* (MTSP).

Since there is a maximum capacity $C \leq I$ of stacks, it is assumed that each pattern has at most C piece types, i.e., $\sum_{i=1}^I p_{ij} \leq C$, for all $j=1,2,\dots,J$. A cutting sequence is given by a permutation π of the columns of P , leading us to matrix $M=\{m_{ij}^p\}$, with $m_{ij}^p = p_{ip^{-1}(x)}$, for $x \in \{1,2,\dots,J\}$, and $m_{i0}^p=0$ by definition. The consecutive-ones property does not necessarily hold for M , for no *fill-ins* are performed after the columns' permutation. We associate a binary $I \times J$ matrix $R=\{r_{ij}^p\}$, such that the following restrictions are met:

$r_{ij}^p \geq m_{ij}^p$ for all $i=1,2,\dots,I$, $j=1,2,\dots,J$ (all the required stacks at the j^{th} -cutting step must be present; matrix R may have fill-ins, as long as it respects the following restriction)

$\sum_{i=1}^I r_{ij}^p = C$, for all $j= 1, 2, \dots, J$ (there are C stacks at each cutting instant, it is advantageous to maintain the C stacks until a switch is necessary), and $r_{i0}^p=0$ for all i . Matrix R holds the current (not removed) stacks versus cutting instants relation for the MTSP: $r_{ij}^p=1$ implies and is implied by stack i present at instant j . Using a formulation analogous to Crama et al. [1994], we define the cost function:

$$Z_{MTSP}^p(P) = \sum_{j=1}^J \sum_{i=1}^I r_{i,j}^p (1 - r_{i,j-1}^p) \quad (4)$$

and we minimize it over the set of all possible permutations Γ , i.e., $\min_{p \in \Gamma} Z_{MTSP}^p(P)$.

Bard [1988]; Tang and Denardo [1988]; and Crama et al. [1994] have previously studied this problem. Tang and Denardo [1988] proposed a mathematical formulation for MTSP, and argued that MTSP is NP-Hard. Unfortunately, as pointed out in Yanasse [1997] and Crama et al. [1994], their argument has flaws. Crama et al. [1994] then reduced the problem of *minimal length traveling salesman path on edge graphs* to MTSP, and demonstrated that the problem is in fact NP-Hard (an even simpler proof uses a reduction from *consecutive blocks minimization* [Garey and Johnson, 1979, problem SR17]).

The objective of the MORP is to minimize the maximum order spread, that is, $\min_{p \in \Gamma} Z_{MORP}^p(P)$, where

$$Z_{MORP}^p(P) = \max_{i \in \{1, \dots, I\}} \sum_{j=1}^J q_{ij}^p \quad (5)$$

The MORP is an NP-Hard problem related to *bandwidth minimization* [Yanasse, 1997]. Madsen [1988] has tried to solve the MORP by minimizing the number of discontinuities that arise on a production sequence, that is, $\min_{p \in \Gamma} Z_{MDP}^p(P)$, where

$$Z_{MDP}^p(P) = \sum_{j=1}^J \sum_{i=1}^I m_{i,j}^p (1 - m_{i,j-1}^p) \quad (6)$$

where the J columns of matrix M are obtained by a permutation of the columns of matrix P , as defined previously (note that the associated matrix for the MORP has fill-ins, but not for

the MDP). Madsen's proposal does not always lead to optimal MORP solutions, for these problems – though closely related – are not equivalent. An issue that is not addressed by Madsen [1988] concerns the computational complexity of the MDP. If the MDP is easily solvable then it might be of use in the development of approximation algorithms, or maybe even of fully approximation schemes for the MORP. Unfortunately, we must point out that MDP is NP-Hard; it has been previously studied under the name of *consecutive blocks minimization* [Garey and Johnson, 1979, problem SR17]. Therefore, MDP is as hard to solve as MORP, and cannot be used in the development of polynomial-time approximation algorithms for MORP (also it is of limited practical use to heuristically solve MORP as suggested by Madsen [1988]).

Although MORP, MDP and MTSP are closely related to the MOSP, these problems are not equivalent. However, the following conjectures were posed in [Yanasse, 1997], in an attempt to exploit some special properties that the MOSP may have:

Conjecture 2.1. *There is always a solution that is simultaneously optimal for MOSP and MORP.*

Conjecture 2.2. *There is always a solution that is simultaneously optimal for MOSP and MDP.*

Conjecture 2.3. *There is always a solution that is simultaneously optimal for MORP and MDP.*

Conjecture 2.4. *There is always a solution that is simultaneously optimal for MOSP and MTSP.*

We now turn our attention to these conjectures. We have developed an enumerative approach to search and detect counter-examples to the conjectures. Consider the following problem instances given by matrices P_1 and P_2 . In table 2 the costs associated with each permutation of the columns of these matrices are presented. Note that only half of the possible permutations are enumerated because these problems are symmetric (i.e., the cost of sequence {1,2,3,4} equals that of sequence {4,3,2,1}).

$$P_1 = \begin{bmatrix} 1001 \\ 1010 \\ 1010 \\ 0101 \\ 0101 \\ 0101 \\ 0110 \\ 0011 \\ 0100 \\ 0100 \\ 0010 \\ 0010 \\ 0010 \\ 0001 \end{bmatrix} \quad P_2 = \begin{bmatrix} 1001 \\ 1001 \\ 1010 \\ 1010 \\ 0101 \\ 0101 \\ 0101 \\ 0110 \\ 0110 \\ 0011 \\ 1000 \\ 1000 \\ 0100 \\ 0010 \end{bmatrix}$$

TABLE 2.2. OBJECTIVE FUNCTION VALUES OF PROBLEMS P_1 AND P_2 ACCORDING TO THE PERMUTATIONS OF THE COLUMNS.

<i>Sequence</i>	$Z_{MOSP}^p(P_1)$	$Z_{MORP}^p(P_1)$	$Z_{MOSP}^p(P_2)$	$Z_{MORP}^p(P_2)$	$Z_{MDP}^p(P_2)$
$\pi=\{1,2,3,4\}$	11	3	11	3	21
$\pi=\{1,2,4,3\}$	9	3	10	3	20
$\pi=\{1,3,2,4\}$	8	3	9	3	17
$\pi=\{1,4,2,3\}$	9	3	9	3	17
$\pi=\{1,3,4,2\}$	8	2	8	2	18
$\pi=\{1,4,3,2\}$	10	2	9	2	19
$\pi=\{2,1,3,4\}$	11	3	11	3	21
$\pi=\{2,1,4,3\}$	9	3	11	3	21
$\pi=\{3,1,2,4\}$	8	3	9	3	19
$\pi=\{4,1,2,3\}$	9	3	10	3	20
$\pi=\{3,1,4,2\}$	7	3	9	3	17
$\pi=\{4,1,3,2\}$	10	3	10	3	18

Proposition 2.3. *Conjecture 2.1 is not true.*

Proof. The conjecture does not hold for matrix P_1 .

Proposition 2.4. *Conjecture 2.2 is not true.*

Proof. The conjecture does not hold for matrix P_2 .

Proposition 2.5. *Conjecture 2.3 is not true.*

Proof. The conjecture does not hold for matrix P_2 .

Proposition 2.6. *Conjecture 2.4 is not true.*

Proof. Let an MTSP instance be defined by the parameters $\langle P, C \rangle$, given by the binary $I \times J$ matrix P where $\sum_{i=1}^I p_{ij} = \sum_{i=1}^I p_{ik}$ for all $j \neq k$, and a maximum capacity of stacks $C = \sum_{i=1}^I p_{ij}$ for any $j \in \{1, 2, \dots, J\}$. For this special *full capacity* case of the MTSP, C stacks have to be used at each cutting instant. This means that a stack (tool) can only be kept open (at the tool magazine) at cutting instant j if it is needed in cutting instants $j-1$ and j . Hence, in this particular case of the MTSP, each stack switch implies and is implied by a discontinuity, and the number of stack switches equals the number of discontinuities. Therefore, $Z_{MTSP}^P(P) = Z_{MDP}^P(P)$ for all P in this special case. Hence, if there is a counterexample to conjecture 2.2 based on this special case of the MDP, it follows that this very same sequence is a counterexample to conjecture 2.4. Observe that P_2 satisfies $\sum_{i=1}^I p_{ij} = \sum_{i=1}^I p_{ik}$ for all $j \neq k$ and is a counterexample to conjecture 2.2. Thus, conjecture 2.4 is false.

Note, however, that conjecture 2.4 is true for the special case where $C \geq \min_{p \in \Gamma} Z_{MOSP}^P(P)$ [Yanasse, 1997].

From the previous results and the observations made in the proof of proposition 2.6 we can also conclude that:

Proposition 2.7. *It is not true that there is always a solution that is simultaneously optimal for MORP and MTSP.*

Proof. Consider problem instance P_2 , proposition 2.5 and the fact that MDP and MTSP are equivalent for this instance if $C=6$.

The following conjecture is placed and left open, based on the fact that it is true for the special full capacity case of the MTSP.

Conjecture 2.5. *There is always a solution that is simultaneously optimal for MDP and MTSP.*

We do not know whether these examples are minimal; we are also intrigued with the question of finding the structural properties (contained in P_1 and P_2) that would actually define why an arbitrary matrix is in fact a counterexample to the conjectures.

Another interesting complexity result that has appeared for the GMLP is the following:

Proposition 2.8. *MOSP does not have an absolute approximation algorithm (unless $P=NP$).*

Proof. By contradiction. The hypothesis $P \neq NP$ implies that there is no polynomial-time algorithm to solve the MOSP exactly, but it admits the existence of a polynomial-time absolute approximation algorithm. We show that such an absolute approximation algorithm can be used to solve the MOSP exactly.

Consider an instance of the MOSP, given by a binary $I \times J$ matrix P . Let ABS_{MOSP}^a be an absolute approximation algorithm for the MOSP, such that $ABS_{MOSP}^a(P) \leq OPT_{MOSP}(P) + a$, where $ABS_{MOSP}^a(P)$ denotes the cost of the solution found by ABS_{MOSP}^a in problem instance P ,

$OPT_{MOSP}(P)$ denotes the optimal solution of P , and α is a constant. Given this hypotheses, we will show that $\alpha=0$.

First, we will generate a new matrix P^* , with J columns, and $(a+1)I$ rows; we make $(a+1)$ ‘replicas’ of the original matrix such that:

$$p^*_{qI+i,j} = p_{i,j} \quad \forall i \in \{1,2,\dots,I\}; \forall j \in \{1,2,\dots,J\}; \forall q \in \{0,1,2,\dots, a\}.$$

From the construction of P^* , we have that, for each column sequence of P , if the number of open stacks is x , then the number of open stacks of the same column sequence of P^* is $(a+1)x$. Thus, $OPT_{MOSP}(P^*)=(a+1)OPT_{MOSP}(P)$. And, from our hypothesis, $ABS^a_{MOSP}(P^*) \leq (a+1)OPT_{MOSP}(P) + \alpha$.

Thus, in each replica of matrix P contained in P^* , ABS^a_{MOSP} has found a solution with cost at most $OPT_{MOSP}(P) + \alpha/(a+1)$. But since $\alpha/(a+1)$ must be integer positive, it follows that α must equal zero.

This result may be extended to all problems given in Table 1. With a similar argument, for example, we can prove that

Proposition 2.9. *MTSP does not have an absolute approximation algorithm (unless $P=NP$).*

Proposition 2.10. *MDP does not have an absolute approximation algorithm (unless $P=NP$).*

Note that in the MTSP, the tool capacity C must also be multiplied. Note also that this transformation does not hold for the MORP. This proof has appeared for the GMLP originally in [Deo et al., 1987].

2.4. A NEW CONJECTURE

Consider the following fact, pointed out in Yanasse [1997]: MOSP and MTSP are equivalent if $C=OPT_{MOSP}(P)$. Thus, in this case, if a fixed parameter algorithm $A^k_{MOSP}(P)$ for MOSP (or for GMLP) with parameter fixed at k decides YES to instance P , then in the corresponding MTSP with capacity $C=k$ it is possible to process all stacks in such a way that no stack removal (switch) is performed until each stack is fully completed, and hence, the associated

matrix R holds the consecutive ones property. Thus, the polynomial-time fixed parameter algorithms known to exist due to the work of Downey and Fellows [1995] and Fellows and Langston [1987] may be used to solve the decision version of the MTSP in these special cases. This, coupled with the fact that other related combinatorial width problems such as *minimum cut linear arrangement* [Garey and Johnson, 1979, problem GT44] have proved fixed parameter tractable [Downey and Fellows, 1995], leads us to place the following open conjecture:

Conjecture 2.6. *MTSP $\hat{I}$ FPT, that is, minimization of stack (tool) switches is fixed-parameter tractable.*

Another motivation for the study of conjecture 2.6 comes from the fact that

Proposition 2.11. *If MTSP $\hat{I}$ FPT, then MDP $\hat{I}$ FPT.*

Proof. We hypothesize a polynomial-time fixed parameter algorithm $A_{MTSP}^k(M, C)$, where M is the binary matrix for the MTSP, C is the stack (tool) capacity, and k is the parameter in question. We must show that this implies the existence of an algorithm $A_{MDP}^{k'}(M')$ that decides in polynomial-time whether matrix M' holds a column sequence with k' or less discontinuities. We thus start with an MDP instance $M' = \{m'_{ij}\}$.

Let $C = \max_j \sum_{i=1}^I m'_{ij}$, and compute a gap $G = C.J - \sum_{j=1}^J \sum_{i=1}^I m'_{ij}$.

Create matrix M as follows:

$$m_{ij} = m'_{ij}, \quad \forall i \in \{1, 2, \dots, I\}, \forall j \in \{1, 2, \dots, J\}, \text{ and}$$

rows $I+1, I+2, \dots, I+G$ of M having only a single nonzero element equal to 1. The ones are distributed over the columns in rows $I+1, I+2, \dots, I+G$ in such a way that there are exactly

$C - \sum_{i=1}^I m'_{ij}$ ones on each column. Hence all columns of matrix M have column sum equal to

C .

Let $k=k'+G$.

It is easy to see that the MTSP instance given by $\langle M, C \rangle$ is the full capacity special case discussed earlier, and that $OPT_{MTSP}(M, C) = OPT_{MDP}(M') + G$. Thus, $A_{MTSP}^k(M, C)$ yields YES if and only if $A_{MDP}^{k'}(M')$ also yields YES. Furthermore, this is all done in polynomial-time.

2.5. CHAPTER SUMMARY

In this chapter we presented some theoretical results for the minimization of open stacks problem and other pattern sequencing problems. The computational complexity of MOSP, previously conjectured as NP-Hard, is confirmed. This result is of fundamental importance because, unless the very unlikely case of $P=NP$ turns out to be true, we can discard the possibility of efficient exact algorithms for the solution of the MOSP. In chapters 3, 4 and 6 to follow, we will be suggesting heuristic methods to efficiently explore the vast solution space of the MOSP and obtain high-quality solutions (that may be optimal).

Other theoretical results show that MOSP is fundamentally distinct from MORP, MDP, and MTSP, in the sense that not always there is a simultaneously optimal solution to both problems. These results discard the possibility of using algorithms for these problems to optimally solve the MOSP. Moreover, unless $P=NP$, there is no absolute approximation algorithm for either the MOSP or the related MDP and MTSP.

We have shown that the solution of MOSP is equivalent to solving GMLP. However, this result is restricted to the classical formulations of these problems. Researchers in VLSI design face other restrictions (which were not considered in the original formulation of GMLP), such as gates of varying sizes, or maybe the restrictions arising for the problem of gate matrix partitioning. On the other hand, there are numerous restrictions which have not been considered in the MOSP, but could still be relevant in a cutting stock setting. For instance, consider the saw time, which is a bounded resource, or the size of the stacks, or perhaps the additional constraints that must be considered if one is approaching the problems of cutting pattern generation and sequencing in an integrated manner. These examples show that, as important as the classical formulations of these problems are, they are still limited, as there are additional cost considerations that they do not address.

Other important questions remain open. For instance, the generalization of conjecture 2.5 beyond the special case of full capacity, or the development of polynomial-time relative approximation algorithms (or even of fully polynomial approximation schemes) still remain as open possibilities.

Conjecture 2.6 is also challenging. Of related interest is the question of whether a polynomial-time fixed parameter decision algorithm for MTSP is of any help to construct a fixed parameter solution (we refer the reader to Downey and Fellows [1995]). The interest of this work goes clearly beyond pattern sequencing; there is a family of combinatorial problems equivalent to MOSP studied independently, as depicted in Table 1. Moreover, we note that the pattern sequencing problems considered in this chapter have received increasing attention in the literature. For instance, references Fink and Voss [1999]; Crama et al. [1994]; Faggioli and Bentivoglio [1998]; Foerster and Wäscher [1998]; Linhares et al. [1999]; Djellab et al. [2000] all deal with modern heuristics for the problems considered here.

CHAPTER 3

LINEAR GATE ASSIGNMENT: A PRELIMINARY HEURISTIC PROPOSAL

3.1. INTRODUCTION

In chapter 2 we have dealt with theoretical considerations concerning the complexity of the MOSP and related problems. As we have seen, not only the MOSP, but also the MDP, MTSP, MORP, and GMLP are classified as NP-Hard, and, unless the unlikely hypothesis of $P=NP$ turns out to be true, there should not exist efficient algorithms for their optimal solution. We have decided to approach them, hence, by heuristic methods that search for high-quality solutions, even if these methods cannot guarantee the optimality of these solutions. This may in fact be the only practical alternative for problems of large size. In this chapter, we will approach the GMLP in its most general form, known as *linear gate assignment*.

Linear gate assignment problems arise in many distinct hardware implementations, such as one-dimensional logic arrays, gate matrices, and programmable logic arrays folding [Möhring, 1990]. It is worthwhile, for the sake of completeness, to redefine the problem in this chapter. The goal in VLSI design is to arrange a set of gates (circuit nodes) in an optimal fashion, such that the circuit layout area is minimized. In a symbolic representation (see Fig. 3.1), connections of a gate are realized vertically, and interconnections between gates are realized horizontally by a set of nets.

Two factors combine to determine the layout area: the number of gates, which is constant, and the number of tracks. Since non-overlapping nets can be folded to the same track, the number of required tracks depends on the ordering of the gates. Thus, in order to minimize the overall layout area, one must obtain an optimal gate sequence. The Fig. 3.1 illustrates the

problem. Each gate contains a set of transistors (represented by dots) positioned for interconnection with other gates. The gate numbering and the number of interconnections passing through each gate are given, in Figs. 3.1b-d, at the top and at the bottom of the layouts, respectively. In Fig. 3.1b, the required connections are realized for the gate ordering shown in Fig. 3.1a, resulting in a 7-track circuit. Now, in Fig. 3.1c, under a new gate sequence, there are at most 5 interconnections through the gates, therefore allowing the folding to a new 5-track layout, as in Fig. 3.1d.

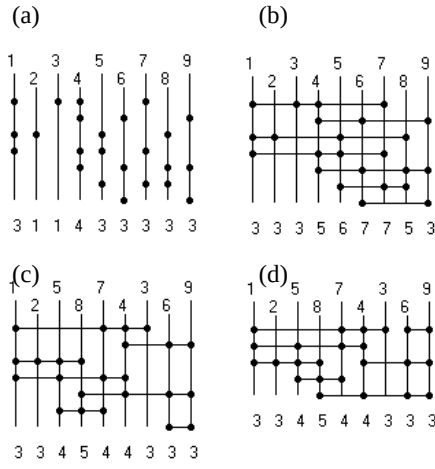


Figure. 3.1. Linear gate assignment.

Note that there are two subproblems: the sequencing of the gates, and the assignment of nets to tracks. There is a polynomial-time algorithm known as the left-edge algorithm for solving the second problem, and thus we are interested only in the sequencing of the gates.

Mathematically, the linear gate assignment problem can be formulated as follows: given a circuit with I nets and J gates, let the binary $I \times J$ matrix, $A = \{a_{ij}\}$, referred to as the *net-gate matrix*, hold the relations between the nets and the gates of the circuit, such that $a_{ij}=1$ if gate j requires a connection to net i , and $a_{ij}=0$, otherwise. A gate sequence can be defined by a permutation π of the gates, and another binary $I \times J$ matrix, $B = \{b_{ij}\}$, can be derived from a column permutation of matrix A , with the additional constraint

$$b_{ij} = \begin{cases} 1, & \text{if } \exists x, \exists y \mid \pi^{-1}(x) \leq j \leq \pi^{-1}(y) \wedge a_{ix} = a_{iy} = 1 \\ 0, & \text{otherwise} \end{cases}$$

where $\pi^{-1}(g)$ is the position that gate g appears in the sequence π .

This constraint relates the gates of each net, and corresponds to the *consecutive-ones property* for matrices [Möhring, 1990; Golumbic, 1980; Ohtsuki et al., 1979]. We can obtain the number of tracks in a layout, by considering the number of interconnections through each gate, which is given by the sum of the corresponding column entries of matrix B . Therefore, the number of required tracks for the layout is given by the maximum column sum of B :

$$Tracks = \max_{j \in \{1..J\}} \sum_{i=1}^I b_{ij} \quad (1)$$

The minimization of the number of tracks - equivalent to *interval graph augmentation* on the vertex versus dominant cliques matrix of a graph, when $I \leq J$ [Ohtsuki et al., 1979] - has long been known to be an NP-Hard problem [Kashiwabara and Fujisawa, 1979]. This means that all known exact algorithms demand a computational effort which is exponential on the size of the problem. Moreover, as discussed in the previous chapter, it has been shown that there is no absolute approximation algorithm for such a problem, unless $P=NP$ – a very unlikely scenario [Deo et al., 1987].

It is interesting to note that other problems with the same objective function as (1) have been independently identified and treated in other industrial contexts. For instance, the problem of sequencing the cutting patterns in the glass-cutting industry, in order to minimize the number of open stacks, or the problem of scheduling a flexible machine, to minimize the number of open client orders [Yanasse, 1997].

In the present work, we consider two layout styles in which the linear gate assignment problem appears: *one-dimensional logic arrays* and *gate matrices* (we note, incidentally, that a similar problem arises in the folding of programmable logic arrays [Möhring, 1990]). In one-dimensional logic arrays, or *Weinberger arrays*, the Boolean functions are produced by

NOR-gates, which are implemented by a linear arrangement of gates [Möhring, 1990; Ohtsuki et al., 1979, Hong et al., 1989]. This is an area-efficient layout style for single MOS, such as NMOS or PMOS. Here, there is one additional constraint: the input and the output are given by boundary gates that must remain fixed on the right and on the left of the circuit, respectively.

In the other layout style, gate matrix layout, developed at Bell Labs [Lopez and Law, 1980], gates are implemented by polysilicon segments which can be interconnected by metal wires (nets). The vertical gates intersecting with the horizontal nets form a matrix that inspires the name of *gate matrix layout* [Hu et al., 1990]. This approach is effective for multilevel combinational functions on large-scale transistor circuits of CMOS technology. As we are focusing here on algorithms for the abstract combinatorial optimization problem, we refer the reader elsewhere [Möhring, 1990; Ohtsuki, 1979; Hong, 1989; Lopez and Law, 1980; Wing et al., 1985] for additional details on these architectures and their underlying technologies.

We have applied a new optimization strategy to linear gate assignment problems arising for the previous layout styles. The considered strategy, called the microcanonical optimization algorithm, is a fast statistical physics alternative to the well-known simulated annealing approach, and has been successfully tested on image processing and computational vision applications, and on the traveling salesman problem [Torreão and Roe, 1995; Linhares and Torreão, 1998, Torreão and Fernandes, 1998]. Our computational results demonstrate the effectiveness of the method for linear gate assignment, as compared to five previous approaches: simulated annealing for one-dimensional logic, the unidirectional and the bidirectional construction methods of [Hong et al., 1989] also for one-dimensional logic, and the artificial intelligence heuristics, GM_Plan and GM_Learn [Hu and Chen, 1990; Chen and Hu, 1990], for gate matrix layout. The microcanonical optimization algorithm has been able to match the results reported for all of these approaches, and for some circuits whose optimal layout is not known, it has even topped the best solutions found so far, by as much as 7 tracks.

Some assumptions underlying the model considered here must be stated upfront: recently, some researchers have introduced algorithms that deal directly with the logic functions for gate matrix layout [Singh and Roger Chen, 1992], generating the layouts directly from the logic equations, instead of obtaining the latter implicitly from the net-gate matrices. This was not attempted in our work; thus, we have assumed that the given net-gate matrix is optimal

with respect to the underlying logic equation. Also, we have not considered the improved net merging method for gate matrix layout introduced in [Shu et al., 1988]. While both of these approaches represent significant advances, our algorithm, if adapted for them, would lose applicability to the one-dimensional logic array layout style.

The remainder of the chapter is organized as follows: in Section 3.2, we treat the microcanonical optimization algorithm, detailing the implementation considered in our work; in Section 3.3, we present our numerical results and the comparisons with previous proposals for one-dimensional logic arrays and for gate matrix layouts. Section 3.4 closes the chapter with a summary of findings.

3.2 SIMULATED ANNEALING AND THE MICROCANONICAL APPROACHES

For the last 15 years, combinatorial optimization problems arising in a wide variety of fields, including computer-aided design, have been approached through statistical mechanics techniques [Kirkpatrick et al., 1983; Barnard, 1989; Hopfield and Tank, 1985]. Such techniques are based on an analogy between the problem of minimization of a multivariable combinatorial function and that of obtaining the ground state (minimum-energy configuration) of a many-particle physical system. Simulated annealing (SA) was the first such technique to be proposed, and still is the most widely applied [Kirkpatrick et al., 1983; Laarhoven and Aarts, 1987]. Its rationale is to simulate the behavior of a physical system as its temperature decreases. It is well known that, at high temperatures, a system can be found equally likely in any one of its available states, while, at sufficiently low temperatures, only the minimum energy states can be reached, a property that is explained by the so-called Gibbs (or Boltzmann) distribution of statistical mechanics. In the physical annealing, in order to obtain a stress-free sample of material, one initially heats it up past its melting point, and then slowly cools it down into a minimum-energy configuration. The simulated annealing algorithm emulates such process, with a non-convex combinatorial functional, which is to be minimized, substituted for the physical energy, and with the temperature playing the role of a global control parameter. At each temperature, solutions are generated as samples from the Gibbs distribution, through the use, for instance, of the computational procedure by Metropolis et al. [1953]. The initial and final annealing temperatures, as well as the cooling

rate, are important implementation parameters, constituting the so-called annealing prescription (see [Geman and Geman, 1984], for instance).

The generality of SA has made it an important tool by which to approach NP-Hard optimization problems. However, it is well known that, even under polynomially-bounded temperature prescriptions, it demands an enormous computational effort, a fact that has spurred the development of faster and more efficient alternatives.

One kind of alternative has appeared through the microcanonical annealing (MA) and microcanonical optimization algorithms (μ O). These, too, are procedures derived from statistical physics, but instead of emulating the behavior of systems under temperature control, they consider thermally-isolated systems. An isolated system does not interact with its environment, and so, has constant energy. Any available state compatible with this fixed-energy condition can then be found with equal probability. To generate samples of such a uniform state distribution, there is a simulation algorithm, the *Creutz algorithm* [Creutz, 1983], which is the microcanonical counterpart to the Metropolis procedure.

In the Creutz algorithm, an internal degree of freedom, referred to as the *demon*, interacts with the system, exchanging energy with it. The demon carries a bag with a variable, but always positive, amount of energy, E_D (with $E_D \ll E_S$, where E_S is the energy of the system), and of maximum capacity E_{DBAG} . The demon/system ensemble is considered isolated, such that its total energy, $E_{TOTAL} = E_S + E_D$, remains constant, despite the exchanges between E_S and E_D .

The demon interacts with the system by proposing random transitions from the current system state to new ones, and executing them according to the energy (*cost*, in optimization) difference involved, ΔE . Transitions are only accepted if ΔE can be either disposed of or received by the demon. Thus, for positive ΔE , only if $\Delta E \leq E_D$ the transition is executed (i.e., the extra energy can be supplied by the demon). On the other hand, if $\Delta E \leq 0$, the condition $E_D - \Delta E \leq E_{DBAG}$ (the released energy can be accommodated in the demon's bag) must be satisfied. In either case, the demon's energy is updated according to $E_D \leftarrow E_D - \Delta E$. In this way, the total energy, $E_S + E_D$, remains constant.

In the Creutz procedure, in contrast to Metropolis', there is no need to generate high-quality random numbers at each iteration, and no transcendental functions are involved. In [Barnard,

1987], a *microcanonical annealing*²¹ algorithm was proposed, based on the iterative application of Creutz's microcanonical simulation for a sequence of decreasing demon capacities, that is to say, with E_{DBAG} following an annealing schedule. Such algorithm is considerably faster than SA, even though it still requires a large number of (energy-based) annealing iterations.

3.2.1. Microcanonical Optimization (μO)

Aiming at further reductions of the computational burden, μO was suggested as an alternative means to explore the advantages of the microcanonical simulation, but without resorting to annealing [Torreão and Roe, 1995; Linhares and Torreão, 1998].

μO consists of two distinct procedures that are alternately applied: *initialization* and *sampling*. The goal of the initialization phase is to implement a fast local search, rapidly converging to a local-minimum solution; from there, the sampling phase takes over, generating alternative solutions at the same cost level through a microcanonical (Creutz) simulation, in order to break free of the local minimum previously attained. After the sampling, a new initialization is run, and the initialization/sampling cycle thus proceeds, until no further improvement will result.

Implementation details concerning the use of μO for linear gate assignment will be given in the following subsection. Here we consider only two further aspects of the initialization and sampling phases:

Initialization phase: The initialization executes a local search, accepting only improving solutions. There are two ways by which this can be implemented: the algorithm may choose the first improving solution, or it may pick the best from a set of neighboring solutions. Our implementation for linear gate assignment followed the former approach.

Sampling phase: in the sampling phase, the algorithm leaves the local-minimum solution obtained in the previous initialization, while remaining close to that (possibly promising) area of the search space. Thus, it implements a microcanonical simulation on the interval $[E_S - E_{\text{DBAG}} + E_I, E_S + E_I]$, for a number of iterations, where E_I is the initial demon energy for that

²¹ It is interesting to point out that an alternative recent approach to simulated annealing in the microcanonical ensemble has come to our notice [Ray and Harris, 1997].

phase (see below). The algorithm may thus accept solutions with cost value as high as $E_S + E_I$ in the sampling, therefore being able to escape from the local optimum solution of cost value E_S obtained in the initialization. In our implementations for linear gate assignment, we took, at each sampling phase, $E_I = E_{DBAG}$. Further implementation details, including the neighborhood mappings and the cost function considered, are given next.

FIG. 3.2. PSEUDOCODE OF MICROCANONICAL OPTIMIZATION.

```

Procedure Microcanonical Optimization;
begin
  numrejected:=0;
  Num_States:=0;
  while (numrejected<5) do
  begin
    Initialization;
    if Cost(Current_Sol)<Cost(Best_Sol) then
    begin
      Best_Sol:= Current_Sol;
      numrejected:=0;
    end
    else numrejected := numrejected+1;
  end;
  Sampling;
end;

procedure Initialization;
begin
  Iter:=0;
  INITPHASE_ITER:=J2/4;
  while (Iter<INITPHASE_ITER) do
  begin
    Iter:= Iter +1;
    Propose_Move;
    if ( $\Delta E < 0$ ) then
    begin
      Num_States:= Num_States +1;
      Accept_Move;
      Iterations:=0;
    end;
  end;
end;

procedure Sampling;
begin
   $E_D = E_{DBAG}$ ;
  for iter:= 1 to SAMPLING_ITER do
  begin
    Propose_Move;
    if ( $\Delta E \leq E_D$ ) and ( $E_D - \Delta E \leq E_{DBAG}$ ) then
    begin
      Num_States:=Num_States+1;
      Accept_Move;
       $E_D := E_D - \Delta E$ ;
    end;
  end;
end;

```

3.2.2. Applying μO to linear gate assignment

Each layout is represented by a permutation, which defines the sequence of gates employed. A move, or possible transition from the current solution to a new one, is performed by selecting two random gates and swapping their positions in the layout. This operation is the same as used in the simulated annealing approach of Hong et al. [1989], and obviously allows one to reach any point in the solution space.

The algorithm starts from random initial solutions. There are no provisions concerning duplicate solutions or solution structures arising from the random number generators.

We have introduced the following cost function to measure the number of tracks of a layout, also considering its wiring length:

$$Cost = IJ \max_{j \in \{1, \dots, J\}} \sum_{i=1}^I b_{ij} + \sum_{j=1}^J \sum_{i=1}^I b_{ij} \quad (2)$$

The first term measures the number of tracks (times IJ), as in (1), while the second measures the wiring length. Since it is more important to optimize the area of the circuit (related to the number of tracks) than to optimize its wiring length, we have given a larger weight to the first term. This is because, since the second term is bound to be less than or equal to IJ , by minimizing the function *Cost* one also minimizes the function *Tracks* of (1), which is the main objective. The secondary goal of minimizing the wiring length, given two layouts with the same number of tracks, is attended by the second term.

Our cost function should be contrasted with the one of Hong et al. [1989], that computes $C = D^2 + \lambda W^2/n$, where D is the sum of the local density of columns (given by number of transistors and of interconnections passing through each gate), W is the sum of the lengths of the nets, n is the number of nets, and the parameter λ controls the relative importance of D and W . Such function purports to attend the same requisites as ours (priority is given to the number of tracks, while still discerning the minor goal of minimizing the wiring length), but on extreme cases it may consider a layout with fewer tracks as worse, depending on the wiring length difference and on the parameter λ . This problem does not occur with our objective function.

The parameters of our microcanonical algorithm have been empirically optimized for speed, and, in contrast to previous applications in other problem domains [Linhares and Torreão, 1998], they were not dynamically changed at run time. Each step of the algorithm consists of an initialization/sampling cycle (see pseudocode in Fig. 2.). The algorithm stops when five such steps have been carried out without solution improvement ($\text{numrejected}=5$).

One iteration of the initialization phase consists of proposing a random transition (a call to `Propose_Move`) and accepting it (by calling `Accept_Move`), when leading to a lower cost. The procedures `Propose_Move` and `Accept_Move` contain the base of the implementation of linear gate assignment problems to the optimization strategies presented above.

The procedure `Propose_Move` selects two random gates G_1 and G_2 (the I/O gates are not selected in the case of one-dimensional logic) and computes the cost difference (ΔE in the pseudocode) incurred if the positions of G_1 and G_2 are exchanged. This value is placed in ΔE . The procedure `Accept_Move` exchanges G_1 and G_2 and updates the cost of the current solution, i.e., $\text{Cost}(\text{Current_Sol}) := \text{Cost}(\text{Current_Sol}) + \Delta E$. Note also from the pseudocode that the number of distinct states visited by the algorithm is counted in variable `Num_States`.

Since the number of possible transitions from each state is $(J^2 - J)/2$, such an order of initialization iterations should always lead to a local minimum solution (as any improving solution will be found). However, we have found out empirically that, with fewer iterations, solutions of comparable quality (but not guaranteed to be local minima) can be reached. Thus, we have made the initialization phase to stop when $J^2/4$ unsuccessful iterations (no improvement over current cost) are counted ($\text{INITPHASE_ITER} = J^2/4$).

One iteration of the sampling phase consists of proposing a random transition and rejecting or accepting it (`Accept_Move`), based on the relation between the demon's current energy and the cost discrepancy of the transition. We have obtained the best results with a small number of iterations, on the order of $\text{SAMPLING_ITER}=25$.

The maximum capacity for the demon, E_{DBAG} , was set at $3.I.J/8$, and this was also the energy initially carried by that variable at the start of each sampling phase ($E_D = E_{\text{DBAG}}$). Given our cost function (2), such an energy is big enough to allow transitions to solutions of lower quality, with regard to the second term (wiring length), but not with regard to the first (number of tracks). Since, going from an n -track layout to an $(n+1)$ -track layout with similar

wiring length incurs the cost difference of IJ , the algorithm must do this gradually, for the demon's initial energy is insufficient to account for the transition in a single sampling step. Therefore, there is a strong pressure to maintain the layout at a reduced number of tracks, as transitions to configurations with more tracks can only be gradually performed over many sampling steps (and thus interrupted by the setting of an initialization phase).

We next present some results of the application of the proposed algorithm to standard circuits, and compare them with those obtained by previous methods.

3.3 NUMERICAL RESULTS

The microcanonical optimization algorithm was coded in PASCAL and executed on a Pentium II - 266MHz processor. In Table 3.1 we present the problem circuits considered in our work, along with the corresponding number of gates, the number of nets, a lower bound to the track number, and the previous best known solution and its reference. The lower bound is obtained by computing the maximum column sum of the original matrix, A . These 30 standard circuits enable us to compare the performance of μO with that of five previously tested approaches: simulated annealing, the bidirectional and the unidirectional approaches to one-dimensional logic arrays of [Hong et al., 1989], and the artificial intelligence planning approach and the related learning approach to gate matrix layout of [Hu and Chen, 1990; Chen and Hu, 1990]. The first set of problems, consisting of the one-dimensional logic arrays **Fujii** through **Data VI**, is available in [Hong et al., 1989]³, while the second set refers to gate matrix layout circuits and was compiled in [Hu and Chen, 1990; Chen and Hu, 1990]. The executions over the one-dimensional logic arrays hold the two boundary I/O columns fixed, while that does not occur for the gate matrix layout circuits.

In Subsection 3.3.1 that follows we compare our results for one-dimensional logic arrays with those reported in [Hong et al., 1989]. In Subsection 3.3.2 a set of computational comparisons, this time with the results reported in [Hu and Chen, 1990; Chen and Hu, 1990] for gate matrix layout is also provided. In Subsection 3.3.3 the performance of microcanonical optimization and of microcanonical annealing algorithms are contrasted.

³ There are some naming conflicts for the test problems of ref. [Hong et al., 1989]. In that reference, the data presented in table II conflicts with that given on fig. 13 and in the appendix. In this chapter, we consider the names given on table II of [Hong et al., 1989] as correct. Thus, fig.13 should read "data I", instead of "Data II", and the appendix presents the netlists

Finally, in subsection 3.3.4 the data obtained on massive computational experiments (1000 runs of the algorithm on selected circuits), on which solutions topping the best known so far have been found, are provided.

In the next subsection, we present results for some one-dimensional logic array circuits. Since, in [Hong et al., 1989], data for a simulated annealing algorithm is provided, it is natural to start our computational comparisons with the results presented therein.

TABLE 3.1.

<i>Circuit</i>	<i># Gates</i>	<i># Nets</i>	<i>LB</i>	<i>Previous</i>
Fujii	9	8	4	4 [Hong et al., 1989]
Data I	9	7	4	5 [Hong et al., 1989]
Data III	15	18	6	7 [Hong et al., 1989]
Data V	29	37	7	13 [Hong et al., 1989]
Data VI	48	48	7	11 [Hong et al., 1989]
v4000	17	10	5	6 [Chen and Hu, 1990]
v4050	16	13	5	5 [Chen and Hu, 1990]
v4090	27	23	9	10 [Chen and Hu, 1990]
v4470	47	37	5	11 [Chen and Hu, 1990]
vc1	25	15	9	9 [Chen and Hu, 1990]
vl	8	6	3	3 [Hu and Chen, 1990]
vw1	7	5	4	4 [Hu and Chen, 1990]
vw2	8	8	3	5 [Hu and Chen, 1990]
w1	21	18	4	4 [Hu and Chen, 1990]
w2	33	48	14	14 [Hu and Chen, 1990]
w3	70	84	11	21 [Hu and Chen, 1990]
w4	141	202	18	34 [Chen and Hu, 1990]
wan	7	8	6	6 [Hu and Chen, 1990]
wli	10	11	4	4 [Chen and Hu, 1990]
wsn	25	17	4	8 [Hu and Chen, 1990]
x0	48	40	6	13 [Chen and Hu, 1990]
x1	10	5	2	5 [Chen and Hu, 1990]
x2	15	6	2	6 [Chen and Hu, 1990]
x3	21	7	2	7 [Chen and Hu, 1990]
x4	8	12	2	2 [Chen and Hu, 1990]
x5	16	24	2	2 [Chen and Hu, 1990]
x6	32	49	2	2 [Chen and Hu, 1990]
x7	8	7	3	4 [Chen and Hu, 1990]
x8	16	11	3	4 [Chen and Hu, 1990]
x9	32	19	3	4 [Chen and Hu, 1990]

for “Data III, V, and VI”, instead of “Data III, IV, and V”. It is worthwhile to point out, nevertheless, that these minor

3.3.1 Comparison with the One-Dimensional Logic Array Algorithms of Hong et al. [1989]

In the paper by Hong et. al [1989], two algorithms for one-dimensional logic assignment are presented: a constructive heuristic, and a standard simulated annealing implementation.

The constructive heuristic attempts to build a layout by means of a mathematical construct that tries to minimize the cut of the seed vertex of a topologically-transformed graph (see Hong et al. [1989] for details). It also takes advantage of the *overriding property*: when all the nets of a gate G_1 also belong to a gate G_2 , G_2 is said to override G_1 , and a special processing is performed to assign those gates sequentially. This method is classified as unidirectional or bidirectional, with respect to the sequence of construction of the layout: the unidirectional construction is predetermined and static, while the bidirectional is flexible and varies according to evaluations performed at execution time. Both methods can still be subdivided into two additional classes, but that should not concern us; for the purposes of our computational comparisons, we consider here only the best result obtained by either the unidirectional or the bidirectional method.

The simulated annealing implementation is a standard one, with the following parameters: each temperature step takes $10J^2$ iterations, where J is the number of columns; the temperature decrease is of 15% between steps. The initial temperature is obtained according to an estimation based on the number of nets [Hong et al., 1989]. Finally, the movement neighborhood - the same used in this chapter - is obtained through simple gate swaps.

TABLE 3.2.

	<i>Sim. Annealing</i>		<i>Bidirectional</i>		<i>Unidirectional</i>		<i>Microcanonical Optimization</i>		
<i>Problem</i>	<i>Tracks</i>	<i>Length</i>	<i>Tracks</i>	<i>Length</i>	<i>Tracks</i>	<i>Length</i>	<i>Tracks</i>	<i>Length</i>	<i>States</i>
Fujii	4	24	4	24	4	24	4	24	44
Data I	N/A	N/A	5	26	N/A	N/A	5	22	98
Data III	7	71	7	71	7	74	7	69	78
Data V	13	245	16	292	13	254	13	245	196
Data VI	11	357	13	393	13	436	11	357	405

naming conflicts do not compromise the quality of ref. [Hong et al., 1989].

In Table 3.2, we present the best results obtained over ten runs of microcanonical optimization, compared to the ones reported for simulated annealing and for the constructive heuristics of Hong et al. [1989]. In the table it is reported, for each algorithm, the number of tracks and the wiring length of the final layout obtained. In the small problem **Fujii**, all algorithms have found equivalent layouts. In the largest problems, **Data V** and **Data VI**, μO has matched the layouts of simulated annealing, and both of these methods have outperformed the constructive heuristics. In problem **Data III**, the layout with shortest wiring length was obtained by μO . However, for all these problems, the results found by all methods were very close in terms of solution quality. This may be due to the small size of the circuits tested, whose optimal layouts can be found without a huge computational effort.

Precise running time data is not available for the methods of Hong et al. [1989]. However, there is evidence that their SA implementation on a VAX 11/750 machine is very slow, since they remark that "... the execution time of the proposed [heuristic] algorithm is about 50 times faster than that of the simulated annealing approach" [Hong et al., 1989]. Here is one issue where μO excels. For instance, for all test problems of this set, μO executes in less than a second (0.2 seconds for **Data V** and 0.7 seconds for **Data VI**, the largest circuits). Since direct comparisons between the running times on distinct machines can be misleading, we provide, in Table 3.2, the average number of distinct states (i.e., distinct solutions) visited by the algorithm from its initial random state to its final state, in order to provide a better standard for future comparisons.

Ideally, we would like to compare our one-dimensional logic array results with known optimal solutions, such as those presented by Asano [1982], but, unfortunately, the circuits considered by him are no longer available [Asano, 1998].

Next, some results for gate matrix layout circuits are presented and contrasted to those obtained in Hu and Chen [1990], and Chen and Hu [1990].

3.3.2 Comparison with the Gate Matrix Layout Algorithms of Chen and Hu

Chen and Hu provide two heuristic algorithms for gate matrix layout derived from artificial intelligence. The first procedure, GM_Plan, is a planning algorithm that basically formulates the gate matrix layout problem as a set of goals and subgoals to be achieved [Hu and Chen,

1990]. The other procedure - GM_Learn - is considerably more sophisticated and time-consuming. This heuristic acts on top of GM_Plan, and attempts to dynamically improve its behavior as it learns from experience [Chen and Hu, 1990].

A comparison of μ O with both of these methods is provided in Table 3.3. In the table it is shown the final number of tracks and the computing times for a VAX 11/750 machine reported in Hu and Chen [1990], and in Chen and Hu [1990], along with the minimum number of tracks obtained over ten executions of μ O, and the average running times required (the running times reported with an ‘-’ indicate that the algorithm took less than 0.1 second to execute, on an average of ten trials). It is also presented the average number of states visited by μ O over those runs. In terms of layout quality, μ O outperforms GM_Plan and GM_Learn in three circuits: **v4470**, **w4**, and **x0**, being outperformed by GM_Learn, by only one track, in problem **wli**. For all other problems, where either GM_Plan or GM_Learn have found optimal layouts, μ O has been able to match these results.

TABLE 3.3.

	<i>GM_Plan</i>		<i>GM_Learn</i>		<i>Microcanonical Optimization</i>		
<i>Problem</i>	<i>#Tracks</i>	<i>Time</i>	<i>#Tracks</i>	<i>Time</i>	<i>#Tracks</i>	<i>Time</i>	<i>States</i>
v4000	7	2.2	6	16.7	6	-	1426
v4050	6	1.6	5	13.1	5	-	1326
v4090	11	4.8	10	43.1	10	0.1	6037
v4470	14	9.3	11	73.2	10	0.7	28221
vc1	10	3.9	9	36.1	9	-	4646
vl	3	0.5	3	3.2	3	-	175
vw1	4	0.5	4	3.1	4	-	172
vw2	5	0.7	5	4.9	5	-	209
w1	4	1.9	4	12.8	4	-	2324
w2	14	8.8	14	66.8	14	0.4	13358
w3	21	25.8	(23-25)	N/A	21	3.9	92963
w4	46	105.7	34	N/A	32	61.7	766052
wan	6	0.6	6	4.1	6	-	138
wli	6	0.6	4	6.4	5	-	462
wsn	8	3.2	10	23.6	8	-	3997
x0	15	11.3	13	102.0	11	0.7	27435
x1	5	N/A	5	6.7	5	-	310
x2	6	N/A	6	13.2	6	-	808
x3	7	N/A	7	24.0	7	-	2431
x4	2	N/A	2	3.1	2	-	122
x5	2	N/A	2	8.3	2	-	614
x6	2	N/A	2	24.4	2	-	2658
x7	6	N/A	4	4.9	4	-	163
x8	10	N/A	4	15.5	4	-	610
x9	18	N/A	4	58.6	4	-	3414

Unfortunately, we do not have the number of visited states, iterations, or other comparable data for GM_Plan or GM_Learn. As already mentioned, given the distinct CPU's, direct comparisons between running times can be misleading. Nevertheless, it can be remarked from our results that μO seems to constitute a fast and reliable approach to gate matrix layout - especially when considering the complexity of the task and the quality of the layouts obtained.

The skeptical reader may argue that “running times are not a practical issue for problems like gate matrix layout; any slow (but polynomially bounded) algorithm will do, as long as the final layouts are of high quality.” This is true, but here also lies another advantage of a heuristic like microcanonical optimization: since each execution of μO is distinct, the user can easily trade between running times and final solution quality, to best suit his or her needs. This is illustrated next.

3.3.3 Tradeoff Between Execution Speed and Solution Quality

Since the real challenge of algorithmic design for linear gate assignment problems lies on the quality side, as opposed to the speed side, we have performed some additional experiments, where microcanonical optimization was executed 1000 times on selected problems for which no optimal solutions are known. While on the vast majority of cases the results have mirrored those already presented, on a small set they have been of extremely high quality - consistently beating the previous best by up to seven tracks. Layouts obtained for five problems - **v4000**, **v4470**, **w3**, **w4**, and **x0** - top the best previously known. Those results are presented in Table 3.4. Furthermore, in problem **wli**, for which μO had found, in the initial runs, a 5-track layout - being thus outperformed by GM_Learn's 4-track solution - layouts with 4 tracks were found in 36.3% of the runs in the massive experiment, indicating that the previous failure to find the 4-track layout was not due to any intrinsic limitation of the method, but to an insufficient number of trials. When the number of trials grew to 1000, microcanonical optimization has not only matched all previous results, but has also yielded the very best results for the circuits of Table 3.4. We also present a new lower bound for these circuits (LB_5), which is the best lower bound presented in [Yanasse et al., 1999].

TABLE 3.4.

<i>Problem</i>	LB_1	LB_5	<i>Previous</i>	<i>New</i>
v4000	5	5	6	5
v4470	5	7	11	9
w3	11	14	21	18
w4	18	19	34	27
x0	6	7	13	11

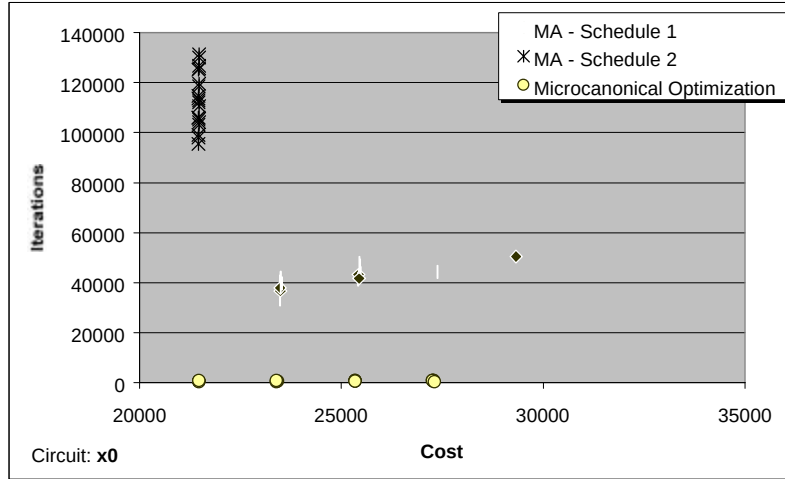
TABLE 3.5.

	MA_1	MA_2
E_{DBAG} decrease rate	0.90	0.96
#iterations at each E_{DBAG}	50	100
Initial E_{DBAG} Value	3IJ	3IJ
# E_{DBAG} decreases	100	100

3.3.4 Comparison with Microcanonical Annealing

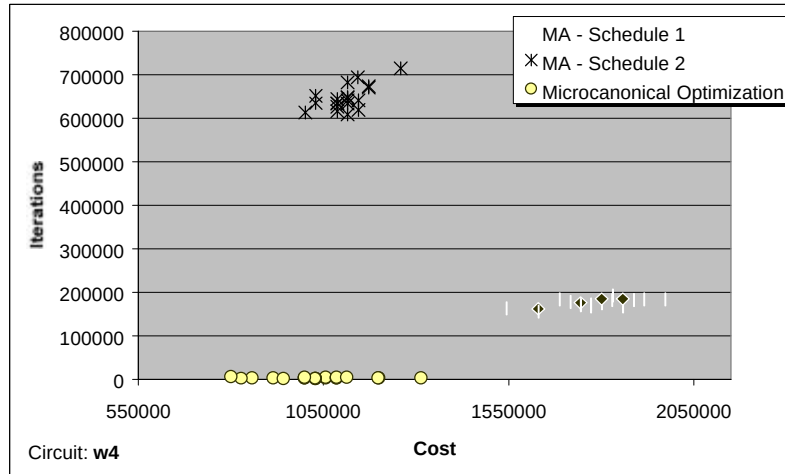
Another optimization strategy that should naturally be compared to μO is microcanonical annealing, the iterated applications of Creutz's algorithm for a monotonically decreasing set of E_{DBAG} values, treated in Section II. In Figs. 3-4, we contrast the performance of μO to those of two distinct microcanonical annealing prescriptions. The first annealing prescription (MA_1) is intended to minimize the number of iterations of the algorithm (i.e., running time), while the second prescription (MA_2), intended to minimize the final solution quality, is much slower. A description of microcanonical annealing is given on [Bhandarkar and Zhang, 1999], and the parameters used by prescriptions MA_1 and MA_2 are given on table 3.5.

FIG. 3.3. COMPARISON TO MICROCANONICAL ANNEALING - CIRCUIT X0.



In figure 3.3, the results obtained over 20 executions of μO , MA1 and MA2 for gate matrix layout circuit x0 are displayed. In the figure the number of iterations (distinct solutions visited by the algorithm) versus the best cost obtained in each of the 20 executions is displayed.

FIG. 3.4 COMPARISON TO MICROCANONICAL ANNEALING - CIRCUIT W4.



Compared to MA₁, μO has generally found higher quality solutions under less computing time. In this circuit, MA₂ has been able to easily find solutions as good as the best found by

μO , but at the expense of running times 217 times longer, on average. In Figure 3.4 the same comparison for the largest circuit on our set, **w4**, is presented. It is clear from the figure that μO outperforms the other methods in both solution quality and running time.

Qualitatively similar results have also been found in other application problems [Torreão and Roe, 1995; Linhares and Torreão, 1998], and there seems to be a plausible explanation for such behavior: in Boese and Kahng [1994], it is argued that monotonically decreasing annealing schedules are optimal if one wants to minimize the expected value of the last solution found by the algorithm given infinite time (referred to as the *where-you-are criterion*). Evidence is presented, on the other hand, that, if one wants to minimize the expected value of the overall best solution found by the algorithm over a finite number of iterations (*best-so-far criterion*), then the annealing schedules may be periodic. The μO algorithm in fact amounts to a periodic-schedule annealing executed over the energy, instead of the temperature, variable. The arguments of Boese and Kahng [1994] present a cogent explanation for the superior performance of μO over the monotonic-schedule microcanonical annealing. Note that a similar approach known as *thermal cycling* has appeared recently (and independently), but it does not take advantage of the faster microcanonical annealing iterations [Möbius et al., 1997].

3.4 CHAPTER SUMMARY

We have presented an analysis of the application of a fast statistical mechanics approach, the microcanonical optimization algorithm, μO , to linear gate assignment problems. μO is based on the microcanonical simulation procedure of [Creutz, 1983], and has been successfully applied to settings as distinct as machine vision and image-processing reconstructions, and the traveling salesman problem [Torreão and Roe, 1995; Torreão and Fernandes, 1998, Linhares and Torreão, 1998].

In the application to linear gate assignment, on both one-dimensional logic and gate matrix layout styles, the algorithm has proven to be fast and efficient, requiring only a small number of state transitions to yield high quality layouts. In one-dimensional logic array circuits, μO has been able to match all the results obtained previously through simulated annealing and

through the unidirectional and the bidirectional methods of [Hong et al., 1989]. In an initial set of experiments on gate matrix layout circuits, it was also able to either match or outperform the GM_Plan and GM_Learn approaches [Chen and Hu, 1990; Hu and Chen, 1990], in 29 out of 30 circuits. A comparison to two microcanonical annealing prescriptions shows that μO is able to find higher quality solutions while still visiting a lower number of distinct solutions, an empirical result that has been found for other problem domains, and may be explained by the *best-so-far* optimality hypothesis of [Boese and Kahng, 1994]. In another set of experiments, when executed 1000 times over selected problems of unknown optimal solutions, the algorithm has not only matched all of GM_Plan's and GM_Learn's results, but has also, in five cases, topped the previous best layouts so far. For instance, in the large circuit **w4**, a layout with 27 tracks was found, a result that beats, by full 7 tracks, the best layout reported in [Chen and Hu, 1990].

Given such performance, we feel safe to conclude that the microcanonical optimization is an effective approach to linear gate assignment problems. It is appropriate, however, to clearly point out two restrictions to this claim: first, our model assumes that the net-gate matrices taken as input are optimal with respect to the underlying logic equations, a limitation which is shared by almost all previously tested approaches, as, for instance [Möhring, 1990; Ohtsuki et al., 1979; Deo et al., 1987; Wing et al., 1985; Chen and Hu, 1990; Hu and Chen, 1990], but notably not by [Singh, and Roger Chen, 1992]. Second, μO is a heuristic approach and, despite the high quality of the solutions yielded (which may be optimal in certain cases), the only guarantee of global convergence that can be provided is given by lower bounds. This is another kind of limitation shared with most previous approaches [Ohtsuki et al., 1979; Hong et al., 1989; Chen and Hu, 1990; Hu and Chen, 1990; Wing et al., 1985; Shu et al., 1988], with the notable exceptions of some exact - but exponential-time, in the worst case - proposals [Yanasse, 1997; Deo et al., 1987; Asano, 1982]. Nevertheless, none of the exact algorithms can currently solve (in a reasonable time) large circuits such as **w4**. As to the heuristics, on the other hand, we have shown μO to match or top the ones in [Hong et al., 1989; Chen and Hu, 1990; Hu and Chen, 1990]. There are limitations, however, that will be addressed on the chapters to come; we are concerned with the robustness of the method: it took 1000 iterations to find these extremely high quality solutions.

In the next chapter, we will be working towards improving the efficiency of the search, by trying to guide it to more promising areas. Hence, we will be developing distance metrics

that measure precisely the minimum distance between any pair of solutions (permutations). These metrics will then enable us, in chapter 5, to propose a new method that should explore the search space more intelligently.

CHAPTER 4

COORDINATION POLICIES FOR COLLECTIVE SEARCH MODELS

4.1 INTRODUCTION

In the last chapter, a heuristic method for linear gate assignment was suggested. The method explores the solution space in a way known as local search, that is, by moving between adjacent solutions (which are derived from a standard neighborhood operator), and guided by a particular (higher-level) strategy. Though the method is promptly able to find high-quality solutions (in comparison to previous methods), there is still a problem of robustness: in some executions, the method starts in a poor region of the solution space, quickly finds a number of locally-minimum solutions which are of low-quality, and halts without being able to find a solution of high-quality. Thus, at these executions, the proposed method is unable to move to a promising region of the search space, as it is short of *search diversity*.

One implicit tenet that has been taken in the local search literature is that “there is a balance between search intensity and search diversity”. The goal of an intensive search process is to find the best solution contained within a relatively small region, while the goal of search diversity is to sample a number of different regions, in order to make sure that the search

space has been properly explored, and so has the region containing the global optimum (global optima).

Each metaheuristic has a way of dealing with this supposed ‘balance’: simulated annealing goes gradually from pure search diversity (at high temperatures) to pure search intensity (whenever the temperature reaches zero); genetic algorithms start with a diverse, random population, and gradually tend towards intensity, as the system evolves in the direction of a ‘clustered’ set of individuals which are akin to each other, losing great amounts of ‘genetic diversity’. Tabu search utilizes long-term memory structures to trigger either a mode of search diversity or a mode of search intensity (it may alternate between modes, but not have both concurrently)⁴. For additional discussions, see Osman and Kelly [1996] or Voss et al. [1998].

In this chapter, we intend to argue that *search intensity and search diversity are not mutually exclusive desiderata*. Even if this fact is true for methods with a single search process, we will argue that it does not hold for methods that employ multiple collective processes, and that search intensity and diversity can (and should) co-exist in a collective local search model. Towards this end, we will present some *coordination policies*, that is, guiding principles for coordinating the collective search among processes; these principles will enable us to present a new collective search model that is able to explore the solution space more intelligently.

4.2 THE USE OF ‘DISTANCE METRICS’

A combinatorial optimization landscape is defined by the triple (Ω, N, Z) , where Ω is a discrete and finite set of solutions, $N: \Omega \rightarrow 2^\Omega$ is a neighborhood operator, and $Z: \Omega \rightarrow \Re$ is the objective function to be minimized. Reachability under N can be assumed, i.e., for all $x, y \in \Omega$, there exists a sequence $x = s_1, s_2, \dots, s_d = y$, where $s_{i+1} \in N(s_i)$ for all $1 \leq i < d$. If d is the minimum

⁴ Of course, we are referring here to the classic formulations of these heuristics. Even if it is not possible to generalize to the great number of applications that are available in this (increasingly expanding) literature, the reader may still equate with our basic argument that search diversity and search intensity are not seen simultaneously. Note, however, that our claim is not that this should be an advantage in all cases, or that heuristics under this approach are inexorably better than other methods.

number of elements in any possible sequence from x to y , then we say that $D_N(x,y)=d$, that is, the distance from x to y under operator N is d .

Surprisingly, these ‘distance metrics’ have not been used in local search models, the only exception being the so-called *bionomic algorithm* [Christofides, 1994; Maniezzo et al., 1998]. Distances can reveal important facts about a pair of solutions: they give an idea of the work (and of the time) required to move from one solution to the other; they are correlated with the probability that the second solution will be visited by a process that passed through the first; finally, they also give a precise measure of similarity between any two solutions. In fact, there is a lack of an accepted measure of diversity in most modern heuristic models, and these distance metrics could be one such measure. In this thesis, we would like to introduce distance metrics as a tool to enhance the performance of local search methods.

Without a notion of distance, a search process may become unsighted. For example, it is indeed possible for a search process to gravitate towards a large ‘attractor’ in the search space, while always keeping a relatively small distance from a particular solution, thus remaining ‘anchored’ to this region for a long time. Countless movements may be performed while the variation in distance remains negligible. Without a precise measure of distance, it is hard to perceive whether this phenomenon is occurring, or if the process is exploring a broader part of the search space.

Unfortunately, terms such as ‘areas’ or ‘regions’ of the search space are rather fuzzy notions. There are no definite boundaries or demarcations⁵ in the search space, and every solution has the very same number of different neighbors. An area around p may be formally defined as $A(p,S): \Omega \times \{1,2,\dots,n\} \rightarrow 2^\Omega$, where S is the “radius” of the area, hence $x \in A(p,S)$ iff $x \in \Omega \wedge D_N(x,p) \leq S$ (for the selected neighborhood operator N). Thus, talking about two processes sharing ‘the same area’ is a notion relative to the size of the area and to the distance taking place between the processes.

In the discussion concerning ‘search areas’ to follow, it is important to estipulate precisely their size, given by S . For the purposes of our discussion, we estipulate S , and thus what is meant by the term ‘area’, with the following intuition: an area should be small enough such that an intensive search process that reaches it should have high probability of finding the

⁵ This is an elusive problem arising not only in combinatorial search spaces, but also in the metaphysics of pattern perception (see Linhares [2000]).

best solution contained within that area. Thus, in the following sections, it is reasonable to conceive of a search area with radius S equal to 5 movements, but not 25 movements, since, due to the combinatorial explosion, a single intensive search process has a significantly larger probability of finding, given an ‘ordinary landscape’, the best solution lying at 5 movements from the current solution than of finding the best solution lying at 25 movements from the current solution.

4.3 DISTRIBUTION COORDINATION POLICIES

In the following we assume that there is a set of concurrent search processes, that each of these processes tends to explore the search space with high intensity⁶, and that there is a distance metric between solutions that is computable in polynomial-time. As a search process (such as those considered in chapter 3) approaches an area, it tries, during a certain amount of time, to find the best possible solution contained in that area. Thus, each process employs an intensive search. What is original about the actual proposal is that search diversity is also desired, and that it can be simultaneously achieved with the use of specific mechanisms.

Two main ideas can be explored: the first is to implement a ‘boundary’ of access for each process. Thus, while multiple processes are conducting the search, the distance between processes can be computed, and, an artificial ‘boundary’ can protect each process’s area (in order to preserve search diversity), such that, should a process attempt to cross another process’s boundary, this attempted movement should be rejected. The size of this boundary can obviously be made to vary with the size of the problem.

The second idea comes from the proposed ‘big-valley’ hypothesis: a correlation between solution quality and distance to the optimum has been observed for some combinatorial optimization problems, including some sequencing problems, such as the *traveling salesman problem* [Boese, Khang and Muddu, 1994]. This correlation suggests a new possibility for

collective search: areas on which solutions of high-quality have been found may be explored proportionally more than areas on which no solution of high-quality has been found.

We suggest some coordination policies, that is, guiding principles to coordinate the collective search, based on the distances measured between search processes. There are two principal objectives for these policies: (i) to implement search diversity, by restricting access, towards a certain distance, to each process' area, and (ii) to redirect processes that have been unable to find high-quality solutions, by moving them to areas of the search space of perceived high quality (which may be potentially more promising).

4.3.1 Uniform non-overlap

Let us start with the *uniform non-overlap* coordination policy. The idea of this policy is to designate a specific area to each search process; these areas, surrounding the solutions explored by the processes, may be set to a uniform size, and may not be entered by other processes (hence its name). Under this policy there can be no overlapping between these areas, and thus there is a guaranteed distribution of the processes along the search space – this guarantee, it should be mentioned, is not achieved in the classic models of either genetic algorithms, simulated annealing, or tabu search (though the latter may incorporate long-term memory structures devised specifically for search diversification, it cannot provide a formal guarantee that the search will be suitably distributed along the search space).

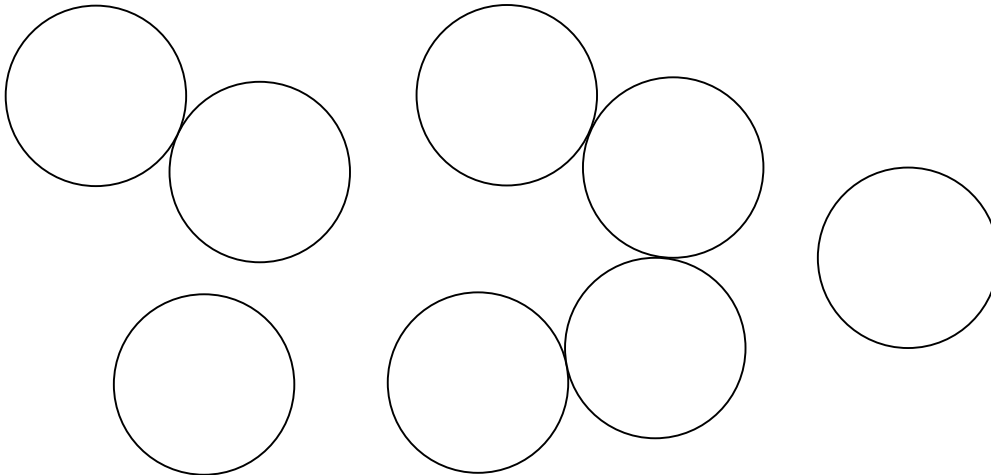
Suppose, for instance, that process P_2 attempts a movement that will bring it into the area demarcated by the radius of process P_1 . In order to prevent this, the movement can be rejected, or, alternatively, process P_1 can be moved outside of the (moving) area corresponding to process P_2 . The 'winning' process can be decided either by the quality of the best solutions found by each process, or by the quality of the current solution under exploration.

There is, however, a high computational cost associated with computing the distance between all pairs of processes – especially when the distance metrics are not computed in linear time.

⁶ For all practical purposes, these search processes could be simple versions of microcanonical optimization, or

One way around this problem is to devise incremental distance metrics, avoiding the entire computation of the distances at each movement, calculating the distances using only the incremental information associated with each operation. This could bring down the complexity considerably. Additional gains can be achieved: to further reduce the computational complexity (for any operator selected), one may opt for reducing the number of distance computations from $O(p^2)$ to $O(p)$. This can be done either by sampling pairs of processes in a random manner, or by adopting one of the coordination policies presented next.

FIG. 4.1. THE UNIFORM NON-OVERLAP COORDINATION POLICY. PROCESSES ARE REQUIRED TO MAINTAIN A MINIMUM DISTANCE TO EACH OTHER (THE BOUNDARIES OF EACH PROCESS ARE REPRESENTED BY CIRCLES).

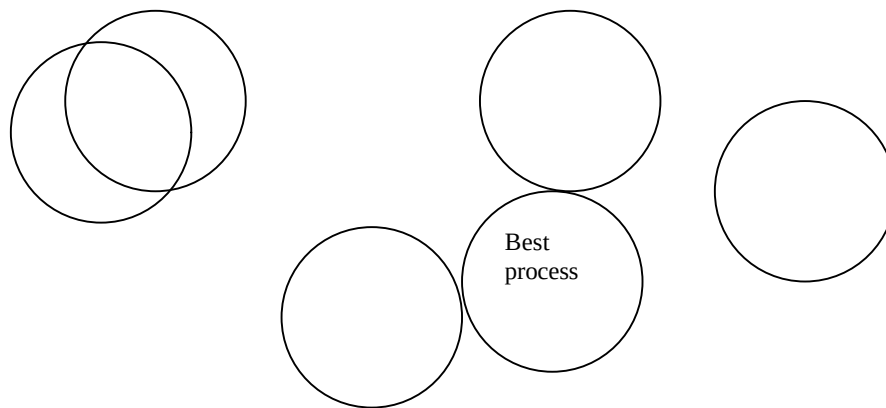


4.3.2 Uniform non-overlap with best only

This is a simple solution to the quadratic complexity presented by the uniform non-overlap policy. Only $O(p)$ distances need to be computed in this model. However, overlapping between any pair of processes that does not include the best one is possible and likely, as

there is no prevention against it. Thus, since overlapping is likely (and hence duplication of search effort), in principle this policy does not appear promising for implementation, but is worthwhile of comment, since it generalizes easily to the next policy, which appears to be more promising.

FIG. 4.2. THE UNIFORM NON-OVERLAP WITH BEST COORDINATION POLICY. PROCESSES ARE REQUIRED TO MAINTAIN A MINIMUM DISTANCE TO THE BEST PROCESS.

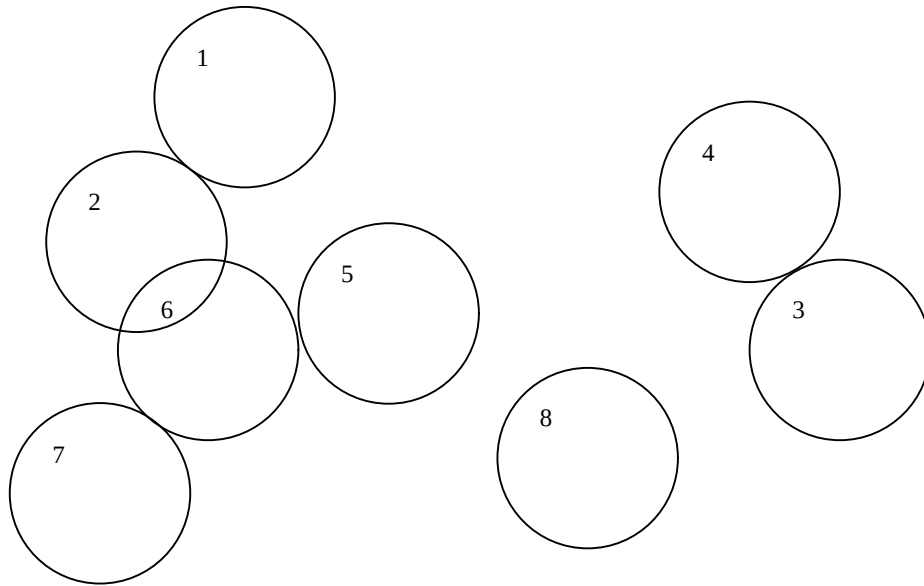


4.3.3 Uniform ordered non-overlap with closest-cost

A new information is considered in this policy: the search processes are ordered, based on the level of quality of the best solution encountered by each process. We then force processes which are 'neighbors' in this ordering (referred to as a 'closest-cost' pair) to refrain from overlapping with one another.

This policy does not enforce a strict guarantee against duplication of effort, but it does indeed place a strong pressure against it, for, as soon as two processes share an area, the chance that these processes will find solutions of similar quality increases. There are two possibilities, then: (1) the processes may find the same solution, or solutions of similar quality, or (2) the processes will find solutions of very distinct quality.

FIG. 4.3. THE UNIFORM ORDERED NON-OVERLAP WITH CLOSEST-COST COORDINATION POLICY. PROCESSES ARE REQUIRED TO MAINTAIN A MINIMUM DISTANCE TO THE CLOSEST-COST PROCESS IN THE ORDER.



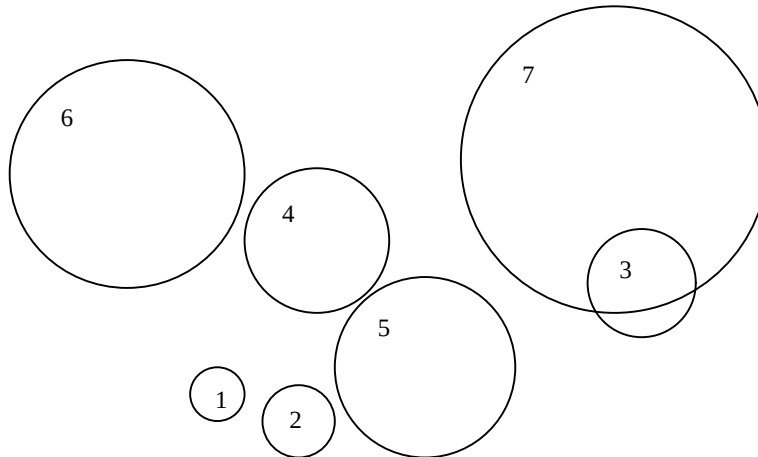
If the processes find solutions of similar level of quality, this will automatically trigger a reordering of processes, and thus there will be high probability that the re-ordered processes will be a closest-cost pair, thus disabling access of the area to one of them, and hence preserving diversity. On the other hand, if the processes do not find solutions of similar level of quality, then this means that the processes have passed through distinct paths (even if their corresponding areas overlapped), and thus diversity is once again guaranteed.

In this policy, there is a further computational burden on ordering the processes, but this should not compromise the performance of the system, for a number of reasons. First of all, the number of processes is usually small. Second, the re-ordering must be done only when a process improves the best solution it has found. Finally, the re-ordering can be done incrementally, by only changing the position of the process that has found a new overall best solution. Hence, the overhead of ordering processes should not place a considerable burden on the system.

4.3.4 Non-Uniform distribution policies

Each of these policies (non-overlap, non-overlap with best only, ordered non-overlap with closest-cost) can also be used in a Non-Uniform region size. That is, the size of the areas allocated to each process can vary according to some specific variable. A candidate for such variable is the cost function obtained by the processes: one may use the order of the processes as a guide for establishing the total size of their areas.

FIG. 4.4. THE NON-UNIFORM ORDERED NON-OVERLAP WITH CLOSEST-COST DISTRIBUTION POLICY. THE BEST SEARCH PROCESSES HAVE INCREASINGLY SMALLER AREAS, WHICH ENABLE AN EXPLORATION OF THE SEARCH SPACE THAT IS PROPORTIONAL TO THE PERCEIVED QUALITY OF EACH AREA, WHILE PERFORMING ONLY $O(p)$ DISTANCE COMPUTATIONS.



In this manner, we consider that the non-uniform ordered non-overlap with closest-cost policy is a prime candidate for distributing search processes over the search space. There is a number of reasons for this. First of all, it has all the advantages of the uniform ordered non-overlapping with closest-cost policy, with the possibility of an increased adaptability. This policy should improve adaptability if the best processes are allowed smaller areas than the worst processes. The regions of the search space which are perceived to contain higher quality solutions are explored in proportion to this perceived quality: a process that has found highest quality solutions holds a small area, and thus enables other processes to explore that area, while the processes occupying the perceived worst areas have a large area size, disabling access to that supposedly inferior quality region.

4.4 RE-DISTRIBUTION COORDINATION POLICIES

The previous policies, with the exception of the non-uniform ordered non-overlap with closest-cost, have the strict objective of maintaining a given distribution of the processes over the search space. In this section, we consider a policy for re-distribution, that is, for redirecting (to another area) a search process that seems to be trapped in a poor region. We explore the idea of concentrating the search effort in the areas of higher perceived quality.

Since we have been considering ordered models (in which processes are ordered, considering the very best solution that was found up to the moment), we may want to redirect the processes that are low on the order (seemingly trapped in poor regions and unable to find high-quality solutions) towards promising regions of the search space. It is undesirable, however, to guide the processes to points that have already been explored. Thus, these processes may be redirected to areas that are closer to good solutions found by the best processes, while still preserving a certain distance, once again for preventing the duplication of effort associated with searching the very same solutions. Of course, this mechanism presupposes a hypothesis that may not be true: that there is a correlation between cost and distance to the optimum solution. This fact may or may not hold for each particular combinatorial optimization problem (and for each specific instance): There is no reason to

discard, at least in principle, the possibility of a landscape where the optimum solution lies surrounded by very low-quality solutions, and it is distant to a great number of high-quality solutions. But one should not ignore the fact that this correlation has indeed been found for a set of problems, and the use of such re-distribution coordination policies should not be discarded *a priori*, for the simple fact that only computational experimentation can decide whether or not the policy improves the search performance for a particular problem.

Let us refer to the current solution considered by the search process that is low on the order as the *source*, and the best solution found by the process that is high on the order as the *target*. There are still two possible variations of such a re-distribution policy: (1) to bring the source closer to the target in one large step, by finding an intermediary solution which belongs to the minimum path between the source and the target; or (2) to do it gradually, i.e., to gradually make the source approach the target, by incrementally computing intermediary solutions (i.e., solutions that belong to the minimum path between the source and the target).

A final test for measuring the distance between the re-distributed process and the remaining processes (which may lie on the same region as they might have been also redirected to that region) could be made. This additional processing should be used if distribution coordination policies are used, in order to enforce diversity.

4.5 CHAPTER SUMMARY

It is generally understood that local search methods must exhibit both search intensity and search diversity in order to be effective to the solution of NP-Hard combinatorial optimization problems. Without search intensity, a method may pass close to the optimum solution, and still be unable to find it. Without search diversity, on the other hand, a method may become trapped in a poor region of the search space, unable to find high-quality solutions. Previous metaheuristic models did not exhibit these desirable goals co-existing simultaneously, and there are numerous cases in the literature of a tacit assumption of a “mutually exclusive balance between search diversity and search intensity”. We argue against this view, presenting coordination policies that can be used for the development of methods that have search diversity and search intensity co-existing simultaneously during the whole course of the process.

With the use of distance metrics, it is possible, for instance, to calculate how much the processes are dispersed over the search space, and, should two (or more) processes cluster around a small area, to drive one (or more) out of that area, in order to eliminate the wasteful duplication of search effort. We have thus considered a number of coordination policies based on the distance information. These policies have a twofold objective: (1) to distribute the processes along the search space, guaranteeing search diversity, and (2) to re-distribute processes that are seemingly trapped in poor regions to more promising regions.

If we assume that each search process is intensive, then we have a collective model where *search intensity and diversity co-exist during the whole course of the search*. In previous models, these desirable goals have been seen to either alternate (GRASP, tabu search, microcanonical optimization), or to move gradually from diversity towards intensity (simulated annealing, genetic algorithms). Other potential advances that were suggested in this chapter are: (i) there can be actual guarantees that there will be no duplication of effort; and, associated to the possibility of a big-valley structure of the landscape, (ii) the added intelligence of the non-uniform ordered non-overlap with closest-cost model, which tends to concentrate the collective search proportionally towards areas with greater perceived quality (measured by the best solution found in each area). Behind these possibilities lies an unexplored territory for modern heuristics research.

The content presented in this chapter could be criticized for not presenting a specific optimization problem and a specific local search strategy. This is as intended, because these ideas are entirely detached from the heuristics underlying the search processes, from the distance metrics employed (which vary from operator to operator), and from the optimization problem being solved, and they thus deserve to be discussed independently. These ideas, taken together, should form the basis, we believe, for the development of new, advanced, collective search strategies.

To illustrate an application of these ideas to the MOSP, we must deal with distance metric algorithms for sequencing problems. Thus, in the following chapter, polynomial-time algorithms to compute the minimum number of applications of a given neighborhood operator between a pair of permutations will be considered, addressing three specific operators: the well-known 2-opt operator for the TSP, the 2-exchange operator, and the insertion operator.

CHAPTER 5

ON DISTANCE METRICS FOR SEQUENCING PROBLEMS: THE 2-OPT, 2-EXCHANGE, AND INSERTION OPERATORS

5.1 INTRODUCTION

In the last chapter, we have considered some mechanisms for improving the robustness of the search, by enabling a search process that is simultaneously diverse and intense. The proposed mechanisms depend on a distance metric to measure the minimum distance between pairs of solutions. This distance is given by the minimum number of applications of some standard neighborhood operator between any two solutions, which in the case of sequencing problems (as used in this thesis) are represented by permutations. We consider the following operators usually employed in combinatorial problems involving permutations: 2-opt (for the traveling salesman problem), 2-exchange, and the insertion operator.

There are two additional motivations for the study presented in this chapter: (i) the recent studies of landscape analysis for which these metrics are readily applicable and, (ii) the problems of comparing genomic sequences in computational molecular biology, which are also related to the problems handled here.

5.1.1 A link with landscape analysis

A new topic that has been recently under study in heuristic search is the landscape of solutions of combinatorial optimization problems. Many researchers have considered the relation involving cost structure and distance, that is, the minimum number of applications of a given neighborhood operator needed to transform a solution into another, between locally optimal solutions. Boese, Kahng and Muddu [1994], in studying the traveling salesman and graph bisection problems, argue for a *big-valley* structure where locally optimum solutions tend to get increasingly closer to each other as they approach the global optimum. New studies followed with similar analysis for the $n/m/P/C_{max}$ flowshop and other problems [Merz and Freisleben, 1999; Reeves, 1999]. Mak and Morton [1995] analyzed the relation between the k-opt and the 2-opt metrics for the traveling salesman problem.

A major drawback of these studies is that the notion of distance has been dealt with approximately. Most authors complain about the lack of known polynomial-time algorithms to measure the minimum distance, in terms of movements, between solutions [Boese et al., 1994; Merz and Freisleben, 1999; Reeves, 1999], and thus resort to approximations of this distance. In this chapter, we reduce this fault proposing polynomial-time algorithms for the 2-exchange and insertion operators for sequencing problems (represented by permutations). For the 2-opt operator for the traveling salesman problem we use Caprara's theorem in computational molecular biology to show that such polynomial algorithm is unlikely to exist.

As an example, consider a 2-exchange move by which two positions in a permutation are exchanged. The computation of the minimum 2-exchange distance between a pair of permutations π and σ may be seen as a shortest path problem on a graph $G=(V,E)$, where each node $v \in V$ is associated with a permutation $\Pi(v)$, and $(u,v) \in E$ if and only if $\Pi(u)$ can be obtained by one 2-exchange operation on $\Pi(v)$. This problem is solvable in principle by the well-known algorithm of Dijkstra [1959]. However, the application of Dijkstra's algorithm would require polynomial-time complexity on the size of G , and the large size of graph G makes this approach intractable, because of the exponential number of permutations Π . We present an alternative linear time algorithm on the size of the permutation that constructs a minimum 2-exchange path from π to σ . We also present a quadratic algorithm for the insertion operator. Before we start to consider the distance metrics, it is worthwhile to notice

that they are related to some recent studies – considered in the next section – in the field of *computational molecular biology*.

5.1.2 A link with computational molecular biology

The study of the minimum number of applications of a given operator between sequences is important in the field of computational molecular biology. This is a third motivation for the study of these problems. In analyzing DNA sequences, for example, it is often found that whole subsequences are found reversed in close species and it is commonly believed that mutations reversing or transposing genome subsequences have generated the evolutionary path between similar species. For example, there are three simple reversals that transform the genome of cabbage into that of turnip [Hannenhalli and Pevzner, 1999]. The goal is therefore to obtain the most likely evolutionary path between the genome sequences, which is given by the minimum number of reversals or transpositions between these sequences [Kececioglu and Sankoff, 1995]. This should provide a good measure of the number of mutations between the species, and may be used as a measure of phylogenetic distance.

In computer science the genome sequences (of same size) appear as permutations $\pi=(\pi_1, \pi_2, \dots, \pi_n)$ and $\sigma=(\sigma_1, \sigma_2, \dots, \sigma_n)$, where π_i and σ_i represent the elements of the permutations, and the objective is to find the minimum number of reversals (transpositions) that transforms π into σ . Note that $D_X(\pi, \sigma) = D_X(\sigma^{-1}\pi, I)$, where σ^{-1} represents the inverse of permutation σ , I represents the identity permutation, and X denotes a standard operator. We may thus consider only the problem of sorting a permutation π with the minimum number of the defined operation. The problem of sorting by reversals, proposed by Kececioglu and Sankoff [Kececioglu and Sankoff, 1995] has received special attention. In that reference, a 2-approximation algorithm and an exact branch and bound algorithm were presented and the problem was conjectured NP-Hard. Another approximation algorithm, with a performance guarantee of 3/2, appeared later [Bafna and Pevzner, 1996] and polynomial time algorithms for some special cases have been recently found [Hannenhalli and Pevzner, 1999; Tran, 1998]. Caprara [1999] recently proved the NP-Hardness of sorting by reversals. Another problem that received attention is sorting by transpositions, that is, sorting with the minimum number of subsequences “jumps” to new locations within the sequence. Bafna and Pevzner found a 3/2-approximation algorithm for this problem [Bafna and Pevzner, 1998] and Gu et

al. [1999] found a 2-approximation algorithm for the related problem of sorting signed permutations by reversals and transpositions⁷.

5.2 COMPLEXITY OF THE 2-OPT DISTANCE METRIC

Because of its wide popularity, the 2-opt distance is definitely the most important metric for the traveling salesman problem. This metric has previously been conjectured to be hard to compute [Kangh et al., 1994; Mak and Morton, 1995]. We must note in passing that the NP-Hardness of computing such a metric follows as a corollary of Caprara's recent result from computational molecular biology.

SORT BY REVERSALS -- SBR

INPUT: Permutation $\pi=(\pi_1, \pi_2, \dots, \pi_n)$

OUTPUT: Minimum number d of reversals $(i_1, j_1), (i_2, j_2), \dots, (i_d, j_d)$ that transforms π in the identity permutation.

Theorem 5.1. (*Caprara's theorem [1999]*) *SORT BY REVERSALS is NP-Hard.*

Proof. Reduction from MAXIMUM ALTERNATING CYCLES IN BREAKPOINT GRAPH.

Corollary 5.1. *Computing the 2-opt distance between TSP solutions is NP-Hard.*

Proof. Reduction from SORT BY REVERSALS.

In a TSP, given the cities p, q, r , and s , a 2-opt move deletes edges pq and rs from the tour, introducing then edges pr and qs . This move corresponds to a reversal of the subsequence of a symmetric TSP tour, such that if we have a sequence $S_1pqS_2rsS_3$, the 2-opt move will

⁷ Signed permutations have, for each element, either a positive or a negative sign. We refer the reader to [Bafna

reverse the inner subsequence to $S_1pr\underline{S_2}qsS_3$, where S_1, S_2, S_3 are sequence fragments and $\underline{S_2}$ is in inverse order.

Thus, for each 2-opt move deleting pq and rs , there is a corresponding reversal that starts at the position occupied by city q on the order in which the cities are visited and ends at the position occupied by city r . Now, construct TSP tours T_1 and T_2 where $T_1 = (\pi_1, \pi_2, \dots, \pi_n)$, and $T_2 = (1, 2, \dots, n)$. Any algorithm for the optimum 2-opt distance will yield an optimum sort by reversals distance. Since sort by reversals is NP-hard, as recently proved by Caprara [1999], computing the optimum 2-opt distance between TSP tours is also NP-Hard. Note that there are good approximation algorithms for SORT BY REVERSALS as pointed out previously in section 5.1.2.

Before we present our algorithms, an observation must be made: a sequencing problem may be either symmetric or asymmetric. It may also be either fixed (start positions make a difference) or cyclic (such as the TSP). The metrics developed in this chapter treat the fixed asymmetric permutations, but can also be used to deal with symmetric and/or cyclic problems. For instance, if the problem is cyclic, such as the *asymmetric traveling salesman problem*, it suffices to force that permutations hold one fixed element. For example, element 1 may be forced to remain fixed in position 1 of the permutation. This simple mechanism not only reduces the solution space but, more importantly, it enables each distinct solution to be represented by one and only one permutation. Similarly, in the case of symmetric permutations, we may force particular elements to always appear in a certain order (for example, element 2 could always appear before element 3), and thus each solution will be represented by a unique permutation. Obviously, these mechanisms may be applied in combination to deal with permutations that are simultaneously cyclic and symmetric.

and Pevzner, 1998] and [Gu et al., 1999] for details.

5.3 THE 2-EXCHANGE OPERATOR

While the 2-opt operator is one of the most used for the TSP, other operators are even more popular for different sequencing problems, such as the 2-exchange and the insertion operators used in many scheduling applications. In this section we present a linear time algorithm that transforms π into σ using the minimum number of 2-exchange operations.

The reader should note that, contrary to what the name suggests, this problem is different from the *optimum exchange sorting* problem studied previously [Knuth, 1975, p.198], in which one is concerned with a comparison-exchange tree, which is a data structure used to study the performance of distinct exchange sorting algorithms, such as quicksort, bubblesort, or mergesort. We refer the reader to [Knuth, 1975] for details.

5.3.1 An algorithm for the 2-exchange distance metric

The 2-exchange operator switches two positions i and j in a permutation, that is, it transforms permutation $(\pi_1, \dots, \pi_i, \dots, \pi_j, \dots, \pi_n)$ to $(\pi_1, \dots, \pi_j, \dots, \pi_i, \dots, \pi_n)$. We present the following linear time algorithm for computing the minimum 2-exchange distance between π and $\sigma=I$.

```
d = 0;
for i= 1 to n do
    if  $\pi_i \neq \sigma_i$  then begin
        2-EXCHANGE( $i, \sigma^{-1}(\pi_i)$  );
        d = d +1;
    end;
return d;
```

This trivial algorithm runs in linear time. We will now prove that it performs the minimum number of 2-exchange movements between π and $\sigma=I$.

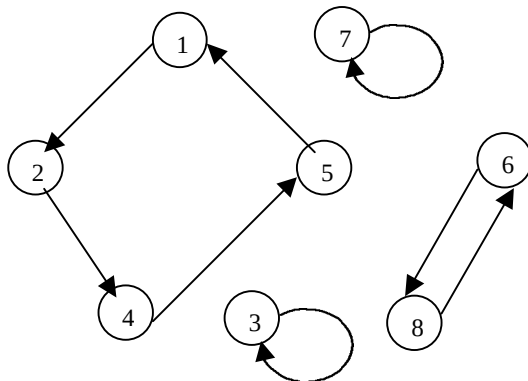
To prove the optimality of the algorithm, we now state the problem in a graph theoretical form we refer to as the switch graph. Let $G=(V,E)$ be a directed graph with vertex set $V=(v_1, v_2, \dots, v_n)$ and edge set E . Let each vertex $v_i \in V$ be associated with a position i in the

permutation π . The edges of G point from position i of π to the position in σ that element π_i appears. That is, directed edge $v_i v_j \in E$ if and only if $j = \sigma^{-1}(\pi_i)$. In figure 5.1 the switch graph for permutations $\pi = (2, 4, 3, 5, 1, 8, 7, 6)$ and $\sigma = I$ is displayed.

Note that, since each vertex sends and receives a directed edge, the switch graph consists of one or more independent cycles. The size of the cycles ranges from $\{1, 2, \dots, n\}$, and the number of distinct cycles can be at most n , in the case of n self-cycles (i.e., vertices pointing to itself). Note that this is precisely the desired goal, i.e., to put each element in its correct position. Thus, we are concerned with increasing the number of cycles with the minimum number of operations possible. The problem can now be stated in terms of the switch graph.

FIG. 5.1. SWITCH GRAPH OF PERMUTATIONS π AND I .

$\pi = (2, 4, 3, 5, 1, 8, 7, 6)$



Let B be the number of cycles in the switch graph between permutations p and s , and consider proposition 5.1.

Proposition 5.1. *Given $B < n$, it is always possible to increase the number of cycles by one unit by means of one 2-exchange movement.*

Proof. Given that $B < n$, it is always possible to break a cycle C_{M+N} of size $M+N$ into two distinct cycles C_M and C_N , for any combination of $M \geq 0$ and $N \geq 0$, in a single exchange operation: select vertices $v_i, v_j \in C_{M+N}$; and, after the 2-exchange operation, the vertices that pointed to v_i and v_j now point to v_j and v_i , respectively. Thus, the cycle C_{M+N} is broken into two distinct cycles C_M and C_N , where M equals the number of edges on the path from v_i to v_j in C_{M+N} , and N equals the number of edges on the path from v_j to v_i in C_{M+N} .

Proposition 5.2. *One 2-exchange movement can increase the number of cycles by at most one unit.*

Proof. When two elements exchange positions in the permutation, 2 edges are deleted from the graph, and 2 edges are inserted. It is necessary to delete 2 edges in order to break a cycle; hence, each movement increases the number of cycles by at most one unit.

Let $T(C_S)$ be the minimum number of applications of the 2-exchange operator to transform any S -sized cycle C_S into S self-cycles. Then, we have

Lemma 5.1. $T(C_S) = S - 1$.

Proof. Follows trivially from propositions 5.1 and 5.2.

Let $D_{2\text{-exchange}}(\pi, \sigma)$ equal the desired 2-exchange distance metric between permutations π and σ . Then, we have

Theorem 5.2. $D_{2\text{-exchange}}(\pi, \sigma) = n - B$.

Proof. It follows from lemma 5.1 that each S -sized cycle needs $(S-1)$ movements to be transformed in S self-cycles. The desired distance metric can be directly obtained by the number of S -sized cycles in the switch graph, measured by

$$D_{2\text{-exchange}}(\pi, \sigma) = \sum_{S=1}^n (S-1)a_S = \sum_{S=1}^n S\alpha_S - \sum_{S=1}^n \alpha_S = n - B, \text{ where } \alpha_S \text{ is the number of } S\text{-sized}$$

cycles between permutations π and σ .

Since the presented algorithm always places one element in its correct position, it always breaks a cycle into two (with the new cycle having size exactly one). This leads to the proof of correctness of the algorithm.

5.4 THE INSERTION OPERATOR

The insertion operator for permutations deletes one element from the permutation, and inserts it in a new position, either between two elements or at the first or last positions, maintaining the order of the other elements intact. Once again, the metric devised here deals with fixed permutations but extension to cyclic problems is readily attainable as, in such case, it should suffice to maintain one element in a fixed position of the permutation (for example, by keeping element 1 in the first position of the permutation).

Note that, despite what the name suggests, the insertion distance metric considered here is not the previously studied number of moves of *sorting by insertion* [Knuth, 1975, section 5.2.1], in which the number of moves refers to the overall running time of the algorithms. We refer the reader to [Knuth, 1975] for details.

In the problem considered by us, it is not trivial to see that the permutation (8,5,6,9,4,2,1,7,3) requires 6 insertions while (1,8,5,2,6,3,9,4,7) requires 4. This problem is very close to that of Aigner and West [1987] for the leading element insertion distance, referred to as *head* insertions, i.e., considering only insertions of the very first element of the permutation (we refer the reader to [Aigner and West, 1987] for details). They have shown that:

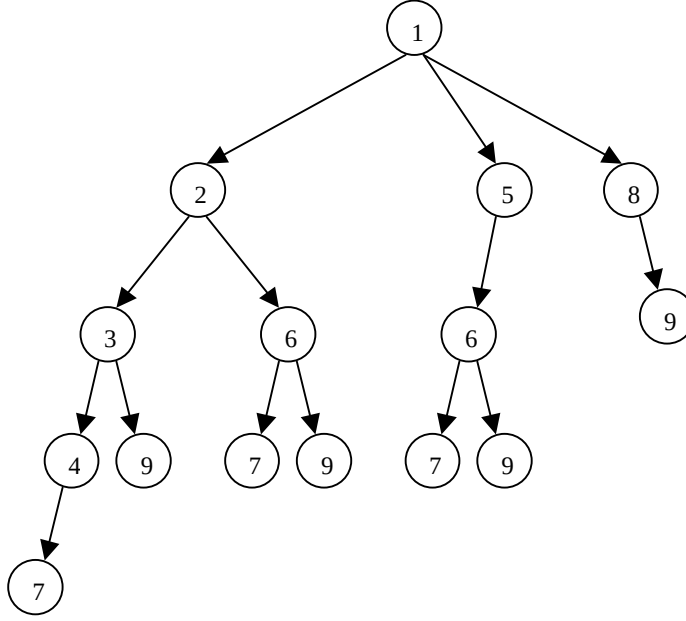
Theorem 5.3. (Aigner and West [1987]). *The number of insertions required to sort a permutation s by head insertions is $n-k$, where k is the largest integer such that the last k entries of s form an increasing subsequence.*

In our more general case, to order by insertion the permutation $(1,8,5,2,6,3,9,4,7)$, for example, the insight needed is that the subsequence $(1,2,3,4,7)$ already appears in the correct order, but the remaining elements are in incorrect positions. We refer to such a subsequence as a *correct subsequence*. A *maximal correct subsequence* is one such that no element of the permutation can be added to it while the subsequence remains in correct order. A maximal correct subsequence with the largest size possible (i.e., maximum number of elements) is denoted as a *maximum correct subsequence*. In this example, the subsequence $(1,5,6,9)$ is a maximal correct subsequence, but is not maximum. The following lemma enables us to establish a link between the insertion distance and the maximum correct subsequence.

Lemma 5.2. *If a permutation contains a maximal correct subsequence of size K , it is always possible to order it in $n-K$ insertions.*

Proof. Relative to the maximal correct subsequence, there are $n-K$ incorrect positions in the permutation. Each one of these positions can be corrected by inserting it in the right position on the maximal correct subsequence by one application of the insertion operator. Thus, $n-K$ insertions are sufficient to order this permutation.

FIG. 5.2. THE TRAVERSAL-TREE OF G FOR $\pi=(1,8,5,2,6,3,9,4,7)$



Thus, given a maximal correct subsequence of size K , $n-K$ insertions are sufficient to order the permutation. (We have not proved, however, that this is the number of necessary insertions to order the permutation.)

It is important to maximize K , i.e., to find a maximum correct subsequence. A maximum correct subsequence can be obtained in polynomial time, as shown next. We will construct a graph (G, V) that may be traversed as a set of trees, where each tree represents one of the maximal correct subsequences. First, create a set of nodes where each node v_i corresponds to a position i in permutation π . Next, create a set E of edges by placing one directed edge $v_i v_j$ iff (1) $i < j$ and, (2) $\pi_i < \pi_j$ and, (3) $\neg \exists k: i < k < j$ and $\pi_i < \pi_k < \pi_j$. Conditions 1 and 2 enforce that positions i and j belong to a correct subsequence while condition 3 avoids transitivity.

Note that G is directed and does not include cycles and may thus be traversed as a set of trees where the nodes that do not receive edges are the roots of the trees. In Figure 5.2, the associated traversal-tree for the example permutation $\pi=(1,8,5,2,6,3,9,4,7)$ is shown.

Lemma 5.3. *The maximum correct subsequence equals the maximum height, H , of all the traversal-trees contained in G .*

Proof. Each path from the nodes to the leaves represents a maximal correct subsequence. Therefore, the maximum correct subsequence will be given by the maximum height of the trees contained in G .

Lemma 5.4. *It is necessary to perform at least $n-H$ insertions.*

Proof. Consider a maximum correct subsequence. If one element of the permutation is deleted, the size of maximum correct subsequence cannot be larger than the original; in fact, it can only remain either equal to the original or equal to the original minus one. If an element is included in the permutation, the size of the maximum correct subsequence cannot be smaller than the original. It can be equal to the original or equal to the original plus one. An insertion movement is equivalent to these two steps being carried out successively. Hence, an insertion movement can augment the maximum correct subsequence in at most one element. As the maximum correct subsequence has size H , it is necessary to carry out at least $n-H$ insertions to order the permutation.

Since we know that $n-H$ insertions are sufficient to order the permutation, this leads us to the desired distance metric:

Theorem 5.4. *The insertion distance of a permutation π of n integers is $n-H$.*

Proof. Follows from lemmas 5.2 and 5.3 and 5.4.

The desired algorithm creates graph G in $O(n^2)$ time, and also traverses the traversal-trees in $O(n^2)$ time.

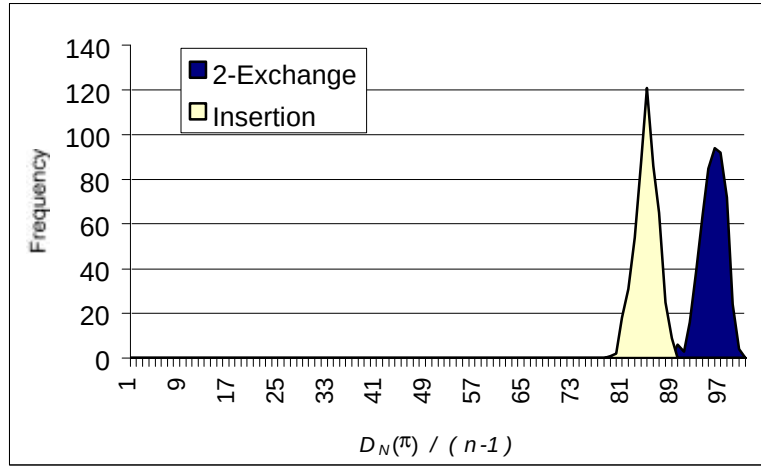
An observation is important here. It is possible to devise incremental algorithms for computing the distance metrics, that is, an algorithm that computes the new distance between two solutions based only on the last movement carried out in one of them, given that the

distance prior to the movement is available. By using such an algorithm, it is also possible to compute intermediary solutions lying along the minimum path between the original pair of solutions. All that has to be done is to construct the proper structure (the switch graph for the 2-exchange operator, or the traversal tree for the insertion operator). The structures promptly indicate which movements minimize the distance: in the case of the switch graph, any movement with edges belonging to the same cycle will break the cycle in two and decrement the distance (see proposition 5.1) and, in the case of the traversal tree, one branch of the tree with maximum height should be selected (in case there are multiple), and all the elements that do not belong to this branch should be inserted at their proper positions. This operation would, at each step, increment the height of the traversal tree and thus, decrement the distance between the previous pair of solutions (see theorem 5.4).

5.5 DISTRIBUTION OF THE INSERTION AND THE 2-EXCHANGE DISTANCE METRICS

In this section we provide data for the distribution of the insertion and the 2-exchange distance metrics. Solutions that are far from each other under one metric may be close under another metric. For example, to transform the sequence $(2,3,4,5,6,\dots,n,1)$ into the identity permutation, it is required to employ $(n-1)$ 2-exchanges versus a single insertion move! Similarly, the sequence $(n,\dots,6,5,4,3,2,1)$ requires $(n-1)$ insertions versus $\lfloor n/2 \rfloor$ 2-exchanges.

FIG. 5.3. DISTRIBUTIONS OF THE INSERTION AND 2-EXCHANGE DISTANCE METRICS.



In Figure 5.3 the frequency of each distance metric, obtained from the distance of 500 random fixed asymmetric permutations to the identity permutation, is displayed. To “smooth” out the distribution, the size of the permutations was made to vary uniformly on the interval [90,110] (The figure retains the same shape if the size of the permutations grows). This figure clearly shows two important points: first, that, from the perspective of any one solution, most solutions lie distant from it; second, that there is a strong statistical tendency for two distinct solutions to be closer under insertion than under 2-exchange operations. This clustering, also found for cyclic and symmetric problems, may suggest that search algorithms might be better served by the insertion operator than by the 2-exchange operator, a hypothesis that should be thoroughly investigated, and is therefore suggested as an open research problem to the field of heuristic search.

5.6 CHAPTER SUMMARY

In this chapter we have introduced some important distance metrics, i.e., algorithms for the precise computation of the minimum number of applications of a given neighborhood operator between a pair of permutations. We presented algorithms for the 2-exchange and

insertion operators, and also note Caprara’s result from computational molecular biology that the distance metric for the 2-opt operator for the TSP is a NP-Hard problem (which closed the conjecture placed by Boese et al. [1994]). These methods may not only be useful for enabling the accurate measurement of combinatorial optimization landscapes as in the recent studies of landscape analysis, but they can actually be used within a higher-level heuristic method, to guide the system towards promising areas of the solution space. It is possible to compute, for example, the distance between the current solution being explored and the best solution found. By employing such metrics, it is also possible to more clearly see whether a search process is exploring many distinct regions of the solution space or if it gravitates around some point. Still another possibility is the development of tabu distances: instead of using lists of previous steps, there could be a “trail” of previously visited solutions and a corresponding distance to be kept from each of these solutions, thus guaranteeing a high search diversity (over the course of the search).

In the following chapter, we will illustrate a new collective model in an application for the MOSP, utilizing the coordination policies and the distance metrics proposed in chapters 4 and 5.

CHAPTER 6

AN ILLUSTRATION OF THE APPLICATION OF A RE-DISTRIBUTION COORDINATION POLICY TO THE MOSP

6.1 INTRODUCTION

The ideas presented in the previous chapters have opened up a number of possibilities for new collective local search models. One of them is the development of collective models that, based on the information generated by the distance between search processes, is able to explore the search space exhibiting a simultaneous co-existence of search intensity and search diversity. Another possibility concerns a search process that, though exploring many regions at once, concentrates the search in proportion to the areas of greater perceived quality, following the suggestion of Boese, Kahng and Muddu [1994], which have, in studying the traveling salesman and graph bisection problems, found a *big-valley* structure where locally optimum solutions tend to get increasingly closer to each other as they approach the global optimum.

These desirable goals can be achieved with the use of the proposed coordination policies. This is in contrast to the previous modern heuristics models, where there are no distribution guarantees. For instance, consider an evolutionary algorithm or a simulated annealing approach, methods which explore numerous distinct regions of the search space at start (with an initial ‘random generation of individuals’ in evolutionary algorithms, or when the temperature is high, in the case of simulated annealing). As these methods evolve, diversity is gradually lost, such that at the end of an execution, there is barely any diversity at all. These models also have no mechanism to intensify the search in proportion to the areas of higher

perceived quality (and thus take advantage of spaces exhibiting a ‘big valley’ structure). As we have seen, these possibilities are promptly within reach for collective search models.

Despite introducing these possibilities, in chapter 4 we have refrained from analyzing any specific application (given by a problem/method combination), in order to properly discuss the many coordination policies available.

In this chapter we illustrate those ideas with an application to the minimization of open stacks problem (MOSP). The MOSP is a sequencing problem, and for this reason in chapter 5 we dealt with some distance metrics for this type of problems. However, it is worthy to mention that the work presented on this chapter is only an illustration of the proposed ideas concerning collective search, based solely on the local cost configuration (i.e., disregarding the special structures that previous studies have demonstrated for the MOSP). We are thus interested in understanding the collective behavior of the search processes and in illustrating the benefits provided by the use of the coordination policies. It is for this reason that we will be considering the effects of coordination policies in the system, but we will not be comparing our results with those obtained by heuristics that exploit special properties of the MOSP.

6.2 SYSTEM ARCHITECTURE

Unfortunately, the current implementation only simulates parallelism: distinct processes alternate their execution at each local search step (i.e., visited solution). It is a desirable research goal to develop a truly parallel algorithm and have it execute asynchronously in a machine with multiple processors, but we must leave such a goal for future research.

The number of concurrent search processes selected for execution in our single-processor case was 20. Obviously, it is desirable to employ a large number of processes, as a greater part of the search space may be explored at once. Also, a larger number of potential interactions between processes will directly influence the collective behavior of the model (and thus test the coordination policies). On the other hand, adding more processes is also desirable, as it affects the performance of the system, which in our single-processor case is seen to gradually degrade to a point of exhaustion, whenever more than 25 processes are executed. Thus, given these tradeoffs, we have selected 20 concurrent processes as the best

cost-benefit parameter for our (simulated parallel) tests. Let us now focus on the dynamics of each individual process.

6.2.1 Single process dynamics

Our local search implementation is similar to that proposed in chapter 3 for linear gate assignment. Solutions are represented by permutations (corresponding to the cutting sequence). New solutions are generated by exchanging two patterns in the permutation: the 2-exchange distance metric will thus be used in the application of the coordination policies. The objective function and our particular evaluation are also as given in chapter 3. The major changes in this chapter come from the use of a coordination policy to guide the collective search processes⁸.

As in chapter 3, the search processes alternate between two search phases: one of intensive search, when the search process is guided towards a local minimum solution, and another termed sampling, when the search is directed towards finding a new region. Let us consider each one in turn.

The intensive search phase is characterized by three aspects: (i) it only accepts improving solutions, since the foremost goal is to obtain a locally-minimum solution; (ii) at each step, it samples 4% of the neighborhood (hence it is a function of the size of the problem) and selects the best sample, as long as it is improving. We have found in experiments that the quality of the results obtained when less than 4% of the neighborhood is sampled decreases considerably, and a complete (or significantly larger) sample places great demand on the execution time, as the evaluation of the cost of a neighboring solution is not an $O(n)$ process [See Fink and Voss, 1999]. This phase stops when $J/2$ subsequent proposed moves are non-improving and are rejected, where J is the number of patterns (corresponding to gates in LGAP or columns in the column permutation).

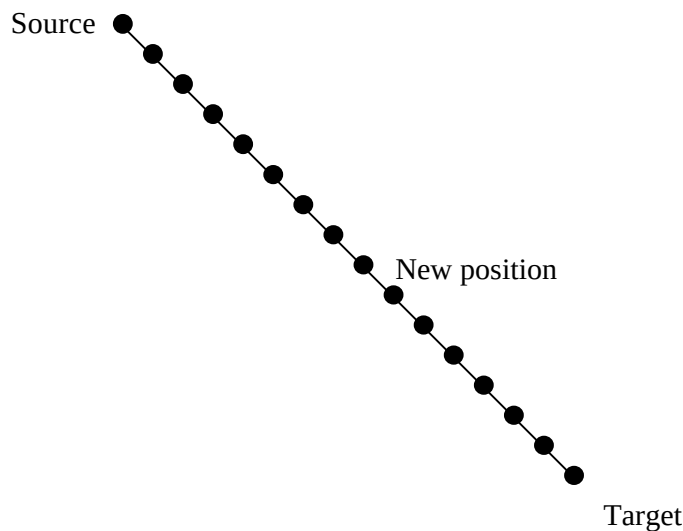
On the other side, in the sampling phase non-improving moves may be accepted if they respect a certain boundary. This boundary in our illustration is set as follows: from the

⁸ The reader may notice that we are not using the term ‘demon’ in this chapter, and that instead we have ‘boundary’. This is to avoid reference to the optimization models in statistical physics, which are not the focus of interest in this chapter. There is no change in meaning, however, and these terms could indeed be interchanged.

current solution, 10 neighboring solutions are evaluated and the 2nd best cost is selected. Since there is a great probability that the last phase stopped at a local minimum solution, these 10 neighboring solutions usually have higher cost than the current solution. Thus, the 2nd best cost among these is a practical value (i.e., known to exist in that context) that is not significantly worse than the current cost, at least in relation to the (random) sample. Thus, there will be a number of solutions that are acceptable from that particular solution (that is, they have better cost than the selected boundary). The process then accepts these solutions, preserving large part of the optimization performed since the start of the execution, while still moving to another region. This phase stops whenever 50 solution movements are attempted; this was usually enough to take the system to a new region.

Each search process was set to stop whenever 10 phases are executed without obtaining any improvement on the cost of the best solution found. The whole algorithm stops whenever the last process stops executing.

FIG 6.1. ILLUSTRATION OF THE RE-DISTRIBUTION COORDINATION POLICY. SOLUTIONS ARE REPRESENTED BY DOTS. A MINIMUM PATH BETWEEN THE SOURCE AND THE TARGET IS COMPUTED, AND THE SOURCE IS RE-POSITIONED AT A SPECIFIC DISTANCE FROM THE TARGET.



6.2.2 Interaction dynamics

A re-distribution coordination policy, intended to concentrate the search over areas of higher perceived quality, is active. Hence, at each $J/3$ steps (measured by the process 1 counter), the current solution of the worst process is selected as the source position, and it is made to approach a target position. This process is illustrated in figure 6.1. The target position was selected randomly from the three best solutions found by the corresponding three most successful processes. A shortest path is computed between the source and the target, and the new current solution of the (previously) worst process was set to approach the target, while remaining at 7 movements of distance from it. This specific parameter was selected (after a number of trials) because it was the smallest number that enabled, in practice, processes to approach targets while subsequently maintaining a small distance (in order to prevent duplication of search effort). As we will argue in the following section, processes do approach each other closely, but at no point does the distance drop to absolute zero (i.e, the wasted effort of passing through the same solutions).

After the re-distribution coordination policy is executed, the processes are re-ordered (based on the best cost achieved by each).

6.3 COLLECTIVE DYNAMICS

In this section, the collective dynamics observed by the multiple processes are studied, and we consider how processes interact during the evolution of the search, with a specific emphasis on the overall distribution of the processes.

The following results have been obtained in the test problem (40,50,0) from Faggioli and Bentivoglio [1998], that is, the first problem (#0) with 40 patterns, and 50 piece types, and they are representative of our larger sample, which is the full set of 300 problems used in Faggioli and Bentivoglio [1998]. These problems were generated randomly and are subdivided in 30 sets of 10 problems. The 30 sets arise from combinations of the number of patterns $J \in \{10,15,20,25,30,40\}$ and the number of piece types $I \in \{10,20,30,40,50\}$, with ten problems being generated for each combination of these values in the following manner: the “composition of every pattern was obtained by randomly extracting four piece types from a uniform distribution over the interval $[1, I]$ ” [Faggioli and Bentivoglio, 1998]. There is also

a final check for completeness, i.e., to ensure that all piece types need to be cut [for details see Faggioli and Bentivoglio, 1998].

As proposed in chapter 4, we have considered the use of distribution and re-distribution coordination policies.

6.3.1 High search diversity

In the previous section the concern was with the development of search processes that present high search intensity. In this section the concern is with the counterpart objective of having high search diversity (by employing a multitude of processes) during the entire execution of the system. Other heuristic search processes may provide mechanisms to support high search diversity as, for example, the long-term memory structures in tabu search. These mechanisms, however, are limited in the sense that there is no *absolute guarantee* of high search diversity and thus, at least in principle, if not in practice, they may not be properly guaranteeing search diversity. In Chapter 4, specific controls were suggested in order to *guarantee* high search diversity. Thus, the distribution coordination policies were suggested for the control of the search processes. However, we feel their use on this particular application is unnecessary, for the reasons presented next.

We have conducted an experiment to understand the natural distribution of the search processes *without interactions between them*, i.e., without using any distribution or re-distribution coordination policies. Our measurements suggest that, under the (previously described) combination of (i) using the 2-exchange operator, (ii) with independent (and intensive) search processes, (iii) in the MOSP search space, then processes tend *naturally to maintain a high distribution over the search space*.

This fact will be clearly seen in the following figures. Given twenty independent search processes, we have calculated the distance from each process to the corresponding closest

process. Each sampling step corresponds to 5 local search steps (measured by the counter of process 1). The plot in Figure 6.2(a) presents the evolution of these numbers over the whole execution of the system. Since the display holds the values for all 20 processes, it is obviously cluttered hence, we will be working with displays such as that of figure 6.2(b), which shows only the corresponding minimum, maximum, and average values corresponding to those given by all processes.

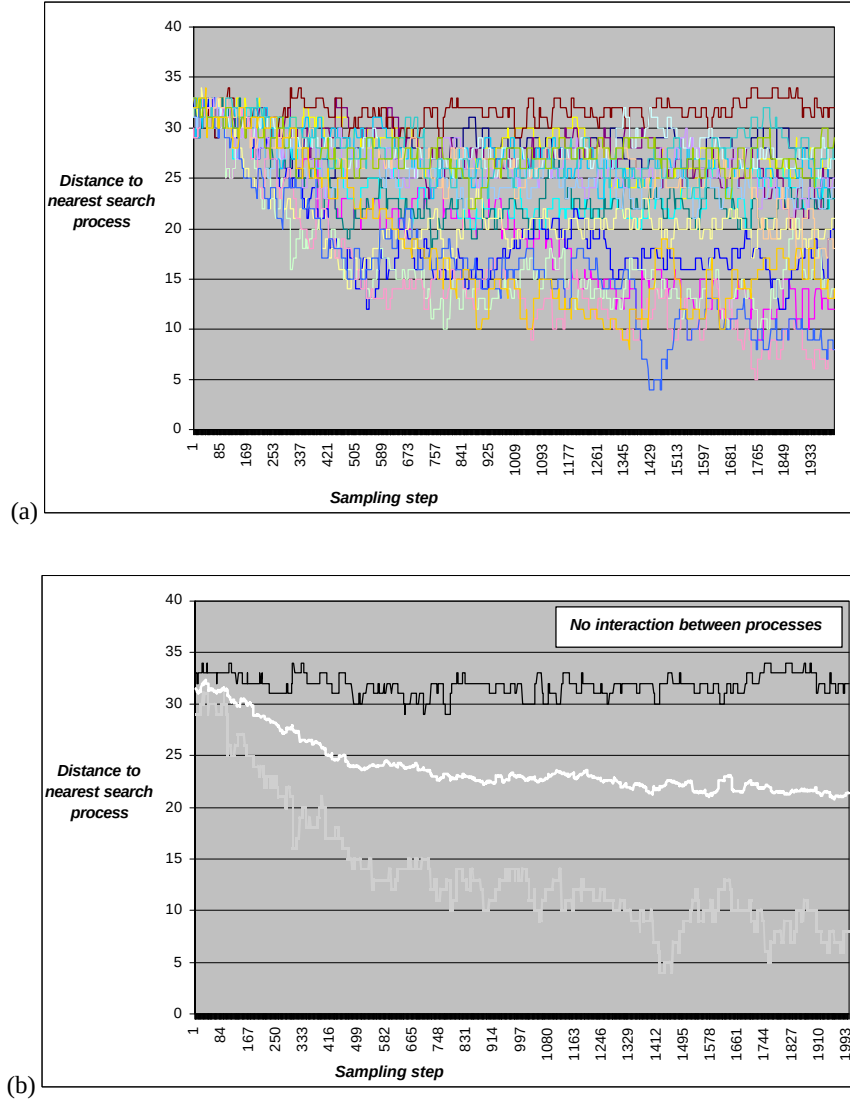
Notice that the average values tend to display a gradual descent over time (starting at around 32 movements, gradually moving towards 25 and, finally, 20 at the end of the run) but, at no point in time, do processes move extremely close to each other, as, for instance, having less than 4 movements of distance. More important, at no point do two distinct processes consider the very same solutions.

Over the course of the whole execution, the minimum distance between closest pairs is 4 moves but the maximum distance *between closest pairs* reaches more than 30 movements. And if we consider other, non-closest, pairs of processes, the distances increase dramatically. For example, in figure 6.3 in which the corresponding graph for the *farthest pairs* of processes is presented, it is clear that processes are widely distributed throughout the search space as the average is higher than 35 moves (while the maximum attainable in this landscape is 39 movements).

Similar results have been reproduced in all the other test problems considered and they lead us to argue for the following observation:

Given our model for the MOSP, the presented independent search processes and the 2-exchange operator, it is unnecessary to enable distribution coordination policies in order to enforce high diversity as the independent processes naturally refrain from clustering (though it is possible that they may closely approach each other).

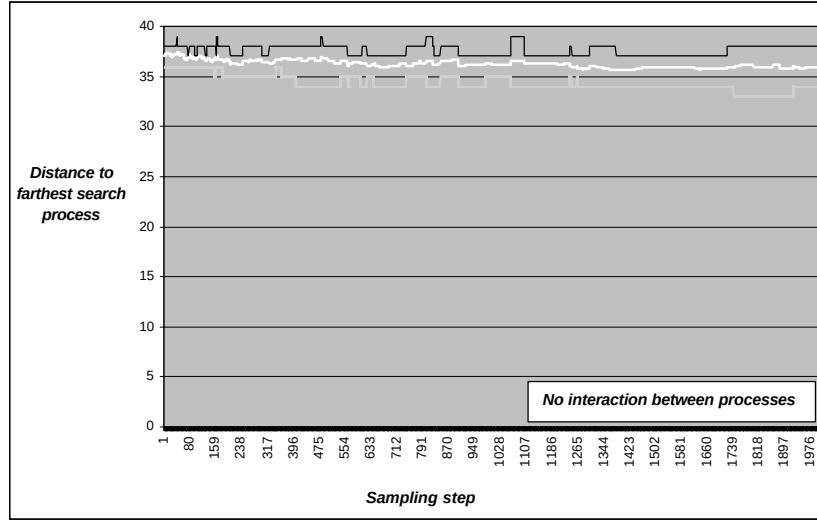
FIGURE 6.2. DISTANCES WHEN PROCESSES ARE INDEPENDENT, I.E., PROCESSES ARE ALLOWED TO NATURALLY DISTRIBUTE THEMSELVES OVER THE SEARCH SPACE. PART (A) DISTANCE OF ALL PROCESSES TO THEIR NEAREST COUNTERPARTS. PART (B) CORRESPONDING MINIMUM CLOSEST-PAIR DISTANCE, AVERAGE CLOSEST-PAIR, AND MAXIMUM CLOSEST-PAIR.



Given a specific implementation, it remains as an empirical question whether processes will cluster and thus have such a wasteful duplication of search effort. Our data demonstrate that this is not the case for this particular application, but it remains to be seen whether this will

hold for other problem domains; at least for this particular case, the previously proposed distribution coordination policies are rendered unnecessary.

FIGURE 6.3. DISTANCE TO THE FARTHEST PROCESS WHEN PROCESSES ARE INDEPENDENT.



There is, however, still potential to improve overall performance by employing a distance-based interaction such as the re-distribution coordination policy proposed in Chapter 4.

6.3.2 Re-distribution coordination policies

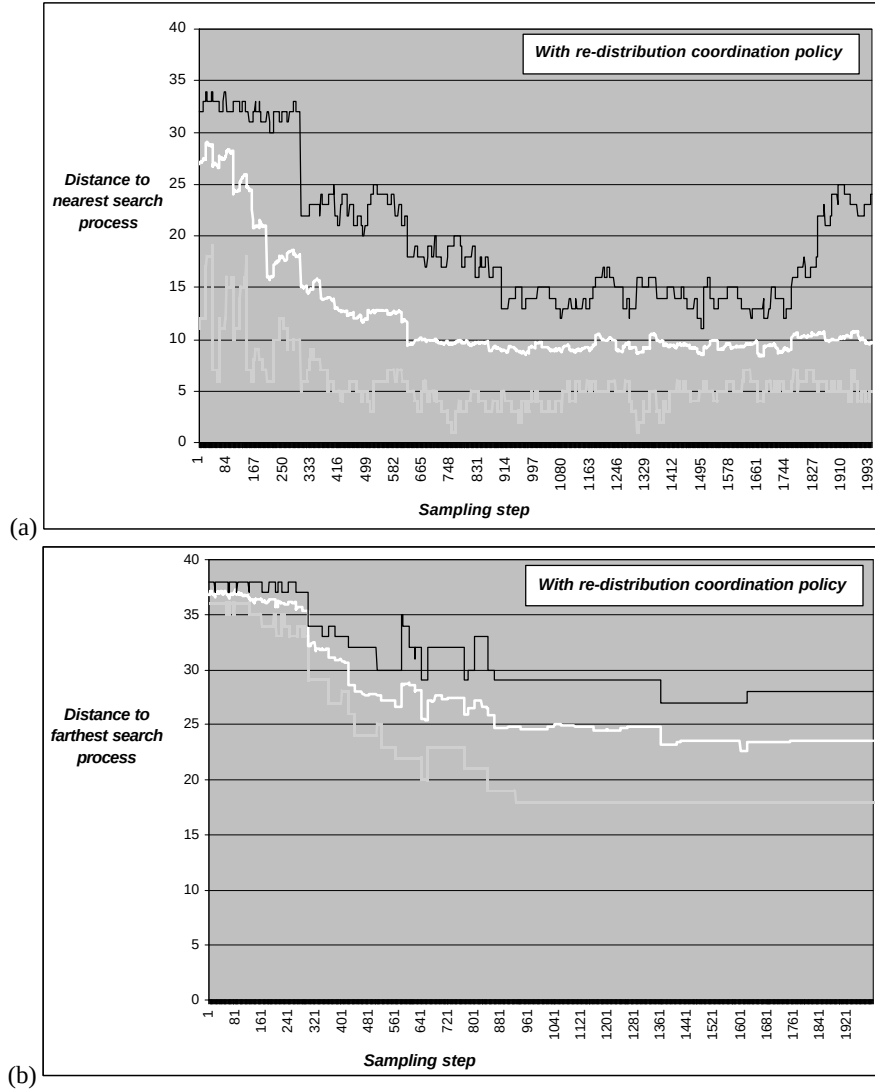
The objective of the re-distribution coordination policy is to redirect the processes that are seemingly trapped in poor regions (and unable to find high-quality solutions) to new areas of the search space. This policy selects potentially good areas by taking advantage of the information obtained by the remaining search processes. As mentioned previously, at $J/3$ local search steps, the worst process is re-directed to a new area by moving it (along a minimum path) towards the best solution found by one of the three most successful processes and leaving it at a particular distance from it (seven movements, in our implementation).

Figure 6.4 is the counterpart to figures 6.2(b) and 6.3, presenting the maximum, average, and minimum values of the distance between each process and its nearest counterpart (in 6.4(a)) or its farthest counterpart (in 6.4(b)). In a direct comparison between figures 6.2(b) and

6.4(a), we may note some facts: (i) as expected, the maximum closest-pair distance drops considerably, and this fact supports the intuition that processes which were in separate (distant) regions are often the processes that have found solutions of worst quality and thus, are the ones selected for re-distribution; (ii) the corresponding average and minimum distances also drop considerably; in this case, the average values, which were between 20 and 25 movements, drop to the neighborhood of 10 movements, while the minimum distances drop from the interval of [5,15] to the tighter interval of [4,7] (and going as low as 2 during two instants). As we have mentioned, seven movements of distance were selected for the implementation of the coordination policy precisely because it was the smallest value that enabled processes to approach targets while subsequently maintaining a small distance (in order to prevent duplication of search effort).

What does this mean for the diversity of the search? Note once again that this is the average of the distance between *closest pairs* of processes, that is, if a process *A* holds distance 5 to its closest process *B*, the distance from *A* to the remaining processes will tend to be larger. Even if the average distance to the nearest process lies on the neighborhood of 10 movements, the average distance to other, non-closest, pairs of processes is significantly larger. For instance, as can be seen in Figure 6.4 (b), which displays the (minimum, average, and maximum) distances to the farthest processes (with interaction), the average distance to the farthest process lies, after the initial convergence, in the interval of 20 to 25 movements. Without interaction, as seen in figure 6.3, the maximum distance lies close to the maximum possible distance, which is, in this case, 39 movements. Even when a redistribution coordination policy is active, the average distance between closest pairs lies around 10 movements and the average distance between farthest pairs lies around 25 movements, with the distance between other pairs of processes generally residing in the interval of [5,30] movements. Notice that, though the processes appear clustered around part of the search space, even this part is still a huge, intractable, solution space. Finally, the high distribution of the search is preserved as all 20 interdependent processes lie on the interval of 10 (average to closest) to 25 movements (average to farthest) in a space with a diameter of 39 movements. This diversity in practice renders irrelevant the inclusion of a distribution coordination policy, as it is not necessary to *enforce* diversity in this application and such overhead can be rightly avoided. Once again, similar results have been obtained for other test problems.

FIGURE 6.4. DISTANCES WITH INTERACTION BETWEEN PROCESSES (RE-DISTRIBUTION COORDINATION POLICY IS ACTIVE). PART (A) DISTANCE BETWEEN CLOSEST PAIRS OF PROCESSES. PART (B) DISTANCE TO FARTHEST PAIRS OF PROCESSES.



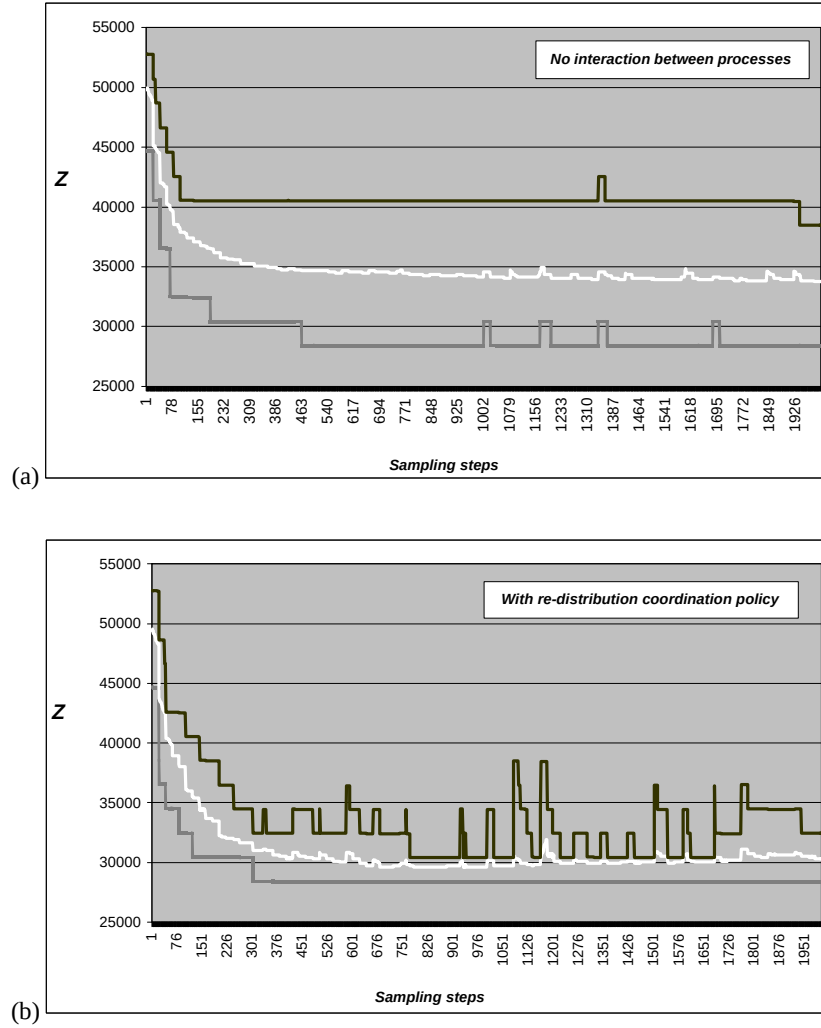
6.4 INTERACTION EFFECTS ON THE COST OF SOLUTIONS

A review of some key points is of worth here as, up to this point, we have only studied the dynamics of the collective system considering how a re-distribution coordination policy affects the distances between processes without, however, taking into consideration the cost of the solutions found. This most significant question has not been dealt with, that is, how does the application of the re-distribution coordination policy affect the cost of the solutions obtained?

In figure 6.5 the evolution of the maximum, minimum, and average cost searched by the multiple processes is shown. Part (a) deals with independent processes (no active coordination policy), while part (b) deals with interacting processes (the re-distribution coordination policy is active). The cost effects can be readily perceived in this graph just as the distance effects were perceivable in the preceding figures. Two significant considerations can be taken from these graphs (they are representative of our larger sample):

- (I) All cost values (the maximum, average, and minimum) obtained by the processes drop dramatically when the policy is active and there is interaction between processes. The maximum values drop considerably more than the others, as expected, for the obvious reason that the worst processes are being moved to new areas (which are close to high quality solutions). As can be seen on the graph, this movement also affects to a large extent the average values and the minimum values converge faster and more robustly (as there is always at least one process in the best known solution).
- (II) Notice that what is under study is the effect of the alteration of search area in the cost obtained by a search process. The re-distribution coordination policy brings processes to a new area (which is closer to one of the best solutions found, while preserving a certain distance from those solutions – seven movements, in our implementation). Processes that have found high quality solutions exercise influence over unsuccessful processes, ‘attracting’ the latter to potentially more promising areas.

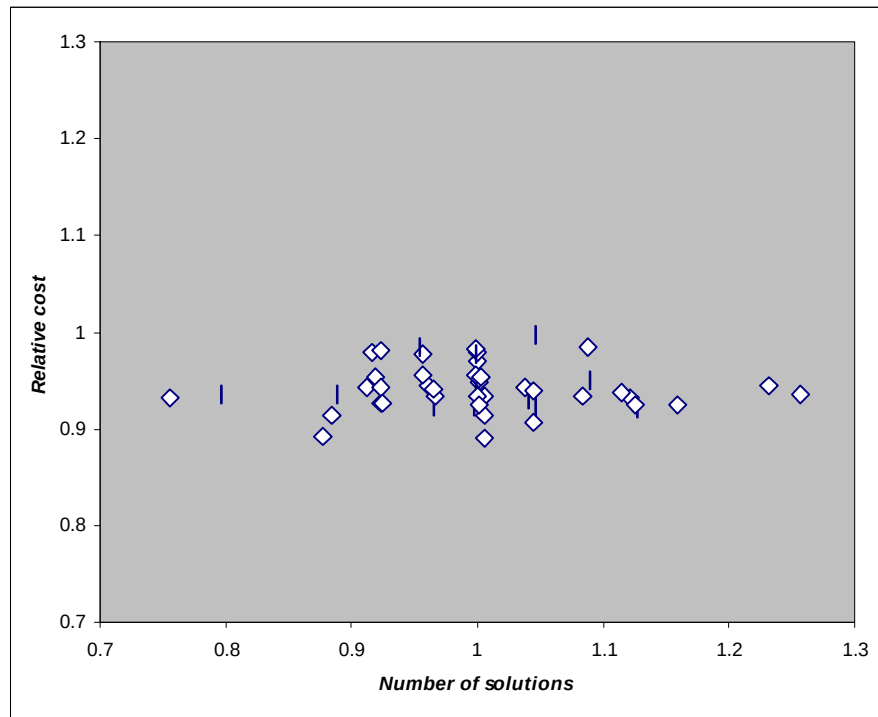
FIGURE 6.5. INTERACTION COST EFFECTS.



The results presented in figure 6.5 are also representative of the other test problems of the set. In figure 6.6 a considerably larger set of problems is considered: the 50 largest problems (with 40 patterns each) considered by Faggioli and Bentivoglio [1998]. Each dot displays the results obtained by the system when the policy is active divided by the results obtained by a

non-interacting system (i.e., the policy is inactive). In this way we can perceive the gains of using the coordination policy *in relation to the results obtained without its use*. Each dot corresponds to (the executions with and without the policy on) a single problem. The axis display the relative number of visited solutions and the relative cost obtained (by averaging the cost of the best processes over a run of the system). Let us start with the relative number of solutions visited.

FIGURE 6.6. AVERAGE COST OF THE BEST SOLUTIONS VISITED AND NUMBER OF SOLUTIONS RELATIVE TO THOSE OBTAINED BY INDEPENDENT PROCESSES WITHOUT THE USE OF RE-DISTRIBUTION COORDINATION POLICY.



In the graph it is clearly shown that the relative number of solutions has a much higher variance than the relative cost of the solutions obtained. Variance for the relative number of solutions is, in fact, 0.008894, and the average of the (number of visited solutions) data is 1.000824, indicating that, on average, *the number of solutions visited when the policy is*

active remains remarkably close to the corresponding number for a system without such policy. The maximum value is 1.25674 while the minimum is 0.755882. This is easy to explain since there is no relation between the coordination policies and the stop criterion used in our system; thus the number of solutions should not increase or decrease with the use of the re-distribution coordination policy.

At the other axis, the relative cost of solutions (computed as the cost of the best processes in tests with re-distribution coordination policy divided by the costs obtained by the best processes of a system without coordination policies) found by the system drops, in average, to 0.942654 of that obtained without the use of the policy. Variance is considerably smaller than that of the number of solutions, at 0.000574. The maximum value is 0.997732 (a marginal gain), and the minimum value is 0.889787 (a significant gain). Note that these results are *in relation to those results obtained by the non-interacting system* which, as we have seen in chapter 3, is by itself capable of finding high-quality solutions and thus what this figure displays, basically, is the *improvement on the quality of solutions found without an increase on the number of solutions visited*, a sign of the increased intelligence of the search generated by the re-distribution coordination policy. This may be true for instances exhibiting a ‘big-valley’ structure where there should be incentive to concentrate the search effort in the areas of higher perceived quality. However, there are instances for which the improvement found is negligible.

Finally, we consider that the previously mentioned goal of increasing robustness of the algorithm has been achieved. For instance, consider the circuit problems dealt with in chapter 3: in problem wli, microcanonical optimization was able to find the optimum solution in only 36.3% of 1000 trials, a disappointing performance, given the relatively small size of the circuit. The collective algorithm presented here has found the optimum in 100% of 100 trials, taking less than a second for each execution. In the considerably larger circuit w3, the collective model is able to find the best known solution of 18 tracks in 50% of 100 trials (averaging 18.67 tracks per run, in 52 seconds of execution). If the re-distribution coordination policy is not active, however, the optimum solution is obtained in only 34 (out of 100) trials.

In the following section our numerical results are compared with those exact and approximate that were presented by Faggioli and Bentivoglio [1998].

6.6 COMPARISON WITH OPTIMAL RESULTS

Three major solution methods are presented in the paper by Faggioli and Bentivoglio [1998]. The first method is an implicit enumeration (a branch and bound approach) that finds optimum solutions (and supposedly takes a large amount of time). The second method is a tabu search procedure, based on an optimized move selection process that, at each step, re-inserts a pattern in the best possible position. The third method is a “generalized local search” procedure that basically works by employing multiple applications of a simplified tabu search that only accepts improving moves; it seems overall to be very competitive with the tabu search. These latter heuristics, the paper concludes, are better alternatives to a number of previous heuristics such as those presented by Yuen [1991,1995] and Yuen and Richardson [1995].

In the columns of table 6.1, it is presented on the following order the number of columns, the number of rows, an average of the optimum value obtained for the ten problems, the corresponding average for our collective method, and for their tabu search and the “generalized local search”. The entries emphasized in bold are deviations from the *reported* optimum value. Observe that some of the entries in the OPT column have value actually higher than we have obtained; we believe that this is due from typos in the [Faggioli and Bentivoglio, 1998] table⁹ and we consider their remaining results as correct (and hence optimum). We have been able to employ the algorithm presented in Yanasse [1997] and solve the set of 15 patterns and 30 items to optimality, obtaining 7.3 open stacks on average¹⁰.

The results clearly show that the method is capable of finding optimum solutions for a large set of problems. In the 30 sets of problems our method was able to produce optimum solutions for at least 25 sets. This means that all the ten problems in each one of the sets have been solved to optimality. Observe also that we are not including the two sets where our results are lower than the reported optima, as we have no proof that these solutions are indeed

⁹ Note that the OPT values for these problems match the reported values for the tabu search method. We believe that this might be the reason for such typos. Anyway, we presuppose that the optimum results hold for the remaining sets of problems.

optimum (we do, however, based on extensive further experimentation, conjecture that such is the case). Thus, should we decide to ignore these three sets of problems, then we can argue that there was a deviation of only 4 additional open stacks over the whole (remaining) set of 280 problems! These results testify for the high quality of the solutions obtained by the collective model.

TABLE 6.1 – COMPARISON WITH OPTIMAL RESULTS

<i>J</i>	<i>I</i>	<i>OPT</i>	<i>Collective</i>	<i>Tabu Search</i>	<i>GLS</i>
10	10	5.5	5.5	5.5	5.5
	20	6.2	6.2	6.2	6.2
	30	6.1	6.1	6.1	6.2
	40	7.7	7.7	7.7	7.7
	50	8.2	8.2	8.2	8.2
15	10	6.6	6.6	6.6	6.6
	20	7.2	7.2	7.2	7.5
	30	7.3 (FB:7.4)	7.3	7.4	7.6
	40	7.3	7.2 (?)	7.3	7.4
	50	7.6	7.4 (?)	7.6	7.6
20	10	7.5	7.5	7.7	7.5
	20	8.5	8.5	8.7	8.6
	30	8.8	9	9.2	8.9
	40	8.6	8.6	8.6	8.7
	50	7.9	7.9	8	8.2
25	10	8	8	8	8
	20	9.8	9.8	9.8	9.9
	30	10.5	10.6	10.7	10.6
	40	10.4	10.4	10.7	10.6
	50	10	10	10.1	10.2
30	10	7.8	7.8	7.8	7.8
	20	11.1	11.2	11.2	11.2
	30	12.2	12.2	12.6	12.2
	40	12.1	12.1	12.6	12.4
	50	11.2	11.2	12	11.8
40	10	8.4	8.4	8.4	8.4
	20	13	13	13.1	13.1
	30	14.5	14.5	14.7	14.6
	40	15	15	15.3	15.3
	50	14.6	14.6	15.3	14.9

¹⁰ This has been done with help from Marcelo S. Limeira. Unfortunately, we have not obtained the corresponding results for the remaining sets of problems.

Moreover, the time needed to compute these solutions was short, with the largest problems taking only 8 seconds. We do, however, make no claims whatsoever in terms of a time comparison with the heuristic methods used in Faggioli and Bentivoglio [1998], for a number of reasons. First, there is not enough information (besides a time-growth curve) presented in the reference to make such a comparison meaningful. Furthermore, the hardware that was utilized there dramatically differs from ours (we do believe that, under ideal conditions, their methods may be faster than ours). However, to emphasize again, our method does not require a significant amount of time (8 seconds for the largest problems, a measure which should even drop as new hardware becomes available). Also, our collective method is naturally prone to a (potentially much faster) parallel implementation.

We believe that a more meaningful comparison should include the number of visited solutions – this measure is independent of hardware and of implementation and can be used even for comparisons between serial and parallel methods. Thus, in table 6.2 we present the average number of solutions visited for each set of problems. It is easy to see that this number grows with the number of patterns but not with the number of piece types, as more piece types do not enlarge the solution space. Interestingly, in four sets of problems, the number of visited solutions of the problems with smallest number of piece types was actually larger than the corresponding number for the problems with largest number of piece types. Notice that the stop criterion does not bear any relation to the number of piece types.

These values are the *totals* for all the 20 search processes and thus, each process, *including the one(s) that found the best solution(s)*, have had, on average, $1/20$ of this number of visited solutions. This brings us to a figure of about 500 visited solutions per process on the largest problems, which contain $40!/2$ distinct solutions. Thus, each process has only a “glimpse” of an incredibly vast search space, but the system still can manage to find, collectively, optimum solutions. As we have mentioned, our implementation of the model only simulates parallelism (and synchronous parallelism at that) but the real gains should come from a truly parallel, asynchronous, concurrent execution of the model in a machine with multiple processors. It remains to be seen whether, in this environment, the processes could visit concurrently such a small number of solutions and the whole system could conclude the search in a fraction of the time it takes to conclude the search in a serial (but simulated parallel) execution, but this is a topic for further research.

TABLE 6.2 – NUMBER OF SOLUTIONS VISITED BY ALL PROCESSES
(AVERAGES TAKEN OVER EACH SET OF PROBLEMS)

		Patterns					
		10	15	20	25	30	40
Pieces	10	1598.0	2940.6	4815.3	5718.0	7353.6	10020.1
	20	1546.1	2902.5	4833.4	5809.9	7262.7	10527.4
	30	1543.0	2923.3	4606.2	5746.1	7383.9	10509.2
	40	1540.2	2931.7	4669.3	6040.6	7614.6	10201.2
	50	1513.9	2930.2	4636.5	5893.4	7588.6	10005.3

6.6 CONCLUSION

In this chapter we have illustrated the use of a re-distribution coordination policy in a MOSP application. In this illustration it is important to precisely spell out what is and what is not being argued about the model. For instance, our conclusions should not suggest that the presented application is the best heuristic currently available for the MOSP. There may be better methods that exploit the special structures exhibited by the MOSP. For example, for any MOSP instance there are other instances that are equivalent to it (that is, they have the same cost), while still having a different structure, such as a distinct number of patterns [Yanasse,1997a]. This is one possibility, among many, of exploiting the particular structure of the MOSP (see also Ashikaga [2001]; Becceneri [1999]). There are also special cases of the problem which can be solved in polynomial time [Yanasse, 1996] – it would be inefficient to approach these special cases with the method presented herein.

Once again, this study is intended to be an illustration of the ideas concerning collective local search methods. The chapter is not, in fact, intended as a major contribution to the MOSP literature but instead as an original contribution for devising innovative local search models

which conduct the search based solely on the local cost configuration, i.e., disregarding any special structures that the problem might have.

Some conclusions may be drawn to validate the study presented in this chapter: We have argued among other things: (i) that the search is simultaneously intensive and distributed and that this is an unusual characteristic which is not observed in most local search models; (ii) that the method is capable of finding optimum solutions (a set of 276 out of 280 problems) in a reasonably short amount of time; (iii) that there is no need, at least for MOSP implementations under the 2-exchange operator, to employ the proposed distribution coordination policies (as processes naturally disperse over the search space) and we have diversity in effect, if not by rule; and finally, (iv) that the use of re-distribution coordination policies enables a more effective and intelligent search process without increasing the number of visited solutions.

Note also that this is just an initial study and many important algorithmic design possibilities remain open. For instance, there was no actual use of the distribution coordination policies, as we have seen, because analysis has shown that, at least in the case of the 2-exchange operator for the MOSP, processes tend to naturally explore distinct regions of the search space. However, this may not be true for other application domains as this structure may be particular to the MOSP. It is important to understand precisely which problems may present a clustering of search processes and thus should be handled with distribution coordination policies (instead of just using a re-distribution coordination policy, as done here). Even if in our case processes do not tend to cluster around small regions this cannot be generalized to other sequencing problems, not to mention other combinatorial optimization problems.

The whole architecture of this collective model is intended for parallel implementations, which can be executed asynchronously over multiple processors. Interesting questions for future research in this line include the interprocess communication that is necessary for implementing the coordination policies.

Another significant possibility that remains open is the use of distance metrics to enforce search distribution over time. The coordination policies presented in chapter 4 have the objective of enforcing high search distribution over the search space at any one instant in time. It is possible to devise analogous mechanisms that enable the search to proceed in a

highly distributed manner, while, of course, employing intensive search processes at each small area. These questions thus remain open to further investigation.

In the next chapter the final conclusions of this thesis will be presented and some extensions for further research will be suggested.

CHAPTER 7

CONCLUSION

In this thesis we have explored some important topics concerning NP-Hard pattern-sequencing problems. The focus was concentrated on the MOSP, especially with regard to its connection to related problems. New heuristic methods based on underlying local search mechanisms were suggested for the solution of pattern sequencing problems, and their performance was substantiated by the results of applications to the LGAP and to the MOSP. Part of this thesis has a strict theoretical focus while another part is empirical. A summary of our findings is presented next.

7.1. SUMMARY OF FINDINGS

In the theoretical side, we have been able to show that MOSP is NP-Hard, moreover, MOSP and LGAP are, under their classical formulations, the very same computational problem. This result demonstrates that the previous algorithms devised for the MOSP can be used for the solution of the LGAP and the converse is also true. This is significant because it establishes a surprising connection between problems arising in these very distinct and previously unrelated areas.

Other significant results show that there can be no absolute approximation algorithm for the MTSP or the MDP (unless, of course, $P=NP$). Other open conjectures posed in [Yanasse, 1997] pertaining to the relation between the MOSP and other pattern sequencing problems such as the MDP, the MTSP, or the MORP, were shown to be false.

The microcanonical optimization algorithm presented in chapter 3 was able to either match or outperform all the previous results obtained by five algorithms previously suggested on the literature: GM_Plan, GM_Learn, simulated annealing for one-dimensional logic and the unidirectional and bi-directional heuristics of Hong et al.[1989]. The algorithm is a viable, competitive, approach to LGAP. A cogent explanation for the superior performance of the method in relation to simulated annealing and to microcanonical annealing was also presented in chapter 3.

We have presented new algorithms for measuring the minimum distance between a pair of permutations. In previous studies, these metrics were estimated (approximately); we have proved that the results obtained by our algorithms, for both the insertion and the 2-exchange distance metrics, are optimal. Moreover, these new methods are polynomial-time bounded and can be used straightforwardly for symmetric and cyclic problems. We have also noted in passing that Caprara's result in computational molecular biology closes the conjecture of [Boese et al., 1994] as the 2-opt distance metric for the traveling salesman problem is also NP-Hard.

Many coordination policies have been proposed to meet two goals: (1) to stimulate the search to promising areas and, (2) to detract from wasted effort of searching the same area twice. These coordination policies have been analyzed in terms of computational overhead imposed by their use. On a more philosophical level, we have attacked the general conception that there is a mutually exclusive 'balance between search intensity and search diversity' and have demonstrated that these two desirable desiderata can indeed take place simultaneously in a collective local search model.

In chapter 6 we have illustrated an application of a new collective model for local search (for the MOSP) where the distance metrics play a major role in coordinating processes. Once again, the method clearly demonstrates that what was once seen as a dichotomy, the so-called 'balance between search intensity and search diversity', is really not a question of a mutually exclusive choice. In our method, search intensity and search diversity occur simultaneously. The computational results demonstrate gains in performance when the distance metrics are used. Other comparisons with the results of [Faggioli and Bentivoglio, 1998] demonstrate that the method regularly finds optimal solutions.

7.2 POSSIBLE EXTENSIONS TO THIS RESEARCH

A vast territory remains uncharted. Some questions are of theoretical nature such as the following conjectures that remain open:

Conjecture 2.5. *There is always a solution that is simultaneously optimal for MDP and MTSP.*

Conjecture 2.6. *MTSP $\hat{=}$ FPT, that is, minimization of stack (tool) switches is fixed-parameter tractable.*

Distance metrics for other operators. One with particular interest is the 2-opt+3-opt operator. This operator is equivalent to employing the following operations: (i) reversal of a subsequence; (ii) transposition of a subsequence, and (iii) reversed transposition of a subsequence. Besides the obvious interest for heuristic search and landscape analysis, this operator should have significant importance on the field of computational molecular biology. This operator would correspond to mutations arising from either of three types of genetic rearrangement: transposition, reversal, and reversed transposition. All of these types of mutations are known to occur regularly in nature, and so, it is proper to study this problem from that perspective. However, we do not feel optimistic about the existence of efficient algorithms, and, though the problem is significantly less restricted than sort by reversals, we conjecture that it is also NP-Hard.

Another interesting uncharted possibility is to use the distance metrics to implement tabu search by employing a ‘tabu distance’ that could keep track of some points previously visited and disabling access to points within a certain distance from them. It is possible to devise on-line algorithms for the distance metrics that would provide instantly (in constant time: $O(1)$, that is) the list of tabu movements. It is also worthwhile to consider the implementation of intensive tabu search processes regulated by the coordination policies considered in chapter 4 and to verify the existence of performance gains with this alternative search strategy.

While it was not possible to conduct an extensive study on the coordination policies proposed, it is still necessary to investigate deeper all the coordination policies available and their dynamics on the collective search models.

Finally, an obvious extension of this work that should be considered is the application of the new collective model to other pattern-sequencing problems such as the MTSP, MDP or MORP. And, as an obvious extension to other sequencing problems, such as job-shop scheduling, or the traveling salesman problem. Still another extrapolation could be the development of distance metrics (or the use of existing ones, such as the *Hamming* distance) to 0-1 or integer problems, which could then be handled by our new collective model.

Combinatorial optimization is a developing field with some questions that still remain impenetrable; the most striking between these being the widely-believed conjecture that $P \neq NP$. Most of this thesis rests upon this assumption. Were it to be proven false, such a result would have tremendous impact on the significance of the theoretical results presented in chapter 2 (which would lose importance, as NP-Hard problems would be easily solvable), and also the very reason for our heuristic proposals in chapters 3 and 6 would lose impact (as soon as a polynomial-time algorithm for solving an NP-Hard problem arises, then, at least in principle, all the problems treated here could be solved *exactly* and *efficiently*). We believe, thus, that $P \neq NP$, and that this conjecture will either be proved, or that it will remain open. Regardless of how this question turns out, we hope to have brought a contribution, over the course of this work, towards the exploration and advancement of a small part of this fascinatingly rich field of combinatorial optimization.

REFERENCES

- Aigner, M., and West, D.B. Sorting by insertion of leading elements. **Journal of Combinatorial Theory Series A**, v.45, p. 306-309, 1987.
- Asano, T. An optimum gate placement algorithm for MOS one-dimensional arrays. **Journal of Digital Systems**, v. 6, pp. 1-27, 1982.
- Asano, T., personal communication, 1998.
- Ashikaga, F.M., Um metodo frugal para o problema de minimização de pilhas abertas. Master's thesis, **Instituto Tecnológico de Aeronáutica**, São José dos Campos, SP, Brazil, 2001.
- Bafna, V., and Pevzner, P.A. Genome rearrangements and sorting by reversals, **SIAM Journal on Computing**, v. 25, p. 272-289, 1996.
- Bafna, V. and Pevzner, P.A. Sorting by transpositions. **SIAM Journal on Discrete Mathematics**, v. 11, p. 224-240, 1998.
- Bard, J.F. A heuristic for minimizing the number of tool switches on a flexible machine. **IIIE Transactions**, v. 20, p. 382-391, 1988.
- Barnard, S.T. Stereo matching by hierarchical, microcanonical annealing. In: Proceedings of the 10th International. Joint Conference on Artificial Intelligence, Milan, Italy, 1987, pp. 832-835.
- Barnard, S.T. Stochastic stereo matching over scale. **International Journal of Computer Vision**, v. 3, p. 17-32, 1989.

- Becceneri, J.C. O problema de sequenciamento de padrões para minimização do número máximo de pilhas abertas em ambientes de corte industriais. Doctoral Thesis, **Instituto Tecnológico de Aeronáutica**, São José dos Campos, Brazil, 1999.
- Bhandarkar, S.M. and H. Zhang. Image segmentation using evolutionary computation. **IEEE Transactions on Evolutionary Computation**, v. 3, p. 1-21, 1999.
- Boese, K.D., Kahng, A.B., and S. Muddu. A new adaptive multi-start technique for combinatorial global optimizations. **Operations Research Letters**, v.16, p.101-113, 1994.
- Boese, K.D. and A.B. Kahng. Best-so-far vs. Where-you-are: implications for optimal finite-time annealing. **Systems and Control Letters**, v. 22, p. 71-78, 1994.
- Caprara, A. Sorting permutations by reversals and Eulerian cycle decompositions. **SIAM Journal on Discrete Mathematics**, v. 12, p. 91-110, 1999.
- Chen, S.J. and Y.H. Hu. GM_Learn: an iterative learning algorithm for CMOS gate matrix layout. **IEE Proceedings E**, v. 137, p. 301-109, 1990.
- Christofides, N. The bionomic algorithm. **AIRO'94 Conference**, Savona, Italy, 1994.
- Crama, Y., Kolen, A.W.J., Oelermans, A.G. and F.C.R. Spijksma. Minimizing the number of tool switches on a flexible machine. **International Journal of Flexible Manufacturing Systems**, v. 6, p. 33-54, 1994.
- Creutz, M. Microcanonical Monte Carlo simulation. **Physical Review Letters**, v. 50, p. 1411-1414, 1983.
- Deo, N. Krishnamoorthy, M.S., and M.A. Langston. Exact and approximate solutions for the gate matrix layout problem. **IEEE Transactions on Computer-Aided Design**, v. 6, p. 79-84, 1987.
- Djellab, H. Djellab, K. and M. Gourgand. A new heuristic based on a hypergraph representation for the tool switching problem. **International Journal of Production Economics**, v. 64, p. 165-176, 2000.

- Dijkstra, E.W. A note on two problems in connection with graphs. **Numerische Mathematik**, v.1, p. 269-271, 1959.
- Downey, R.G. and Fellows M.R. Fixed parameter tractability and completeness. 1. Basic results. **SIAM Journal on Computing**, v.24, p.873-921, 1995.
- Faggioli, E. and C.A. Bentivoglio. Heuristic and exact methods for the cutting sequencing problem. **European Journal of Operational Research**, v. 110, 564-575, 1998.
- Fellows, M.R., and M.A. Langston. Nonconstructive advances in polynomial-time complexity. **Information Processing Letters**, v. 26, 157-162, 1987.
- Fink, A. and S. Voss. Applications of modern heuristic search methods to pattern sequencing problems. **Computers & Operations Research** v. 26, p.17-34, 1999.
- Foerster, H. and G. Wäscher. Simulated annealing for order spread minimization in sequencing cutting patterns. **European Journal of Operational Research**, v.110, p.272-281, 1998.
- Fomin, F.V. Helicopter search problems, bandwidth and pathwidth. **Discrete Applied Mathematics**, v. 85, p. 59-70, 1998.
- Garey, M. and D.S. Johnson. **Computers and Intractability**, Freeman, New York, 1979.
- Geman, S. and D. Geman. Stochastic relaxation, Gibbs distributions, and the Bayesian restoration of images. **IEEE Transactions on Pattern Analysis Machine Intelligence**, v. 6, p. 721-741, 1984.
- Glover, F.W., and M. Laguna. **Tabu search**. Kluwer Academic Publishers, 1998.
- Golumbic, M.C. **Algorithmic graph theory and perfect graphs**, Academic Press, London, 1980.
- Gu, Q.P., Peng, S., and H. Sudborough. A 2-approximation algorithm for genome rearrangements by reversals and transpositions. **Theoretical Computer Science**, v. 210, p. 327-339, 1999.

- Hannenhalli, S. and P.A. Pevzner. Transforming cabbage into turnip: polynomial algorithm for sorting signed permutations by reversals, **Journal of the Association of Computing Machinery**, v. 46, p. 1-27, 1999.
- Hong, Y.S., Park, K.H. and M. Kim. A heuristic for ordering the columns in one-dimensional logic array. **IEEE Transactions on Computer-Aided Design**, v. 8, p. 547-562, 1989.
- Hopfield, J.J. and D.W. Tank. Neural computation of decisions in optimization problems. **Biological Cybernetics**, v. 52, p. 141-152, 1985.
- Hu, Y.H. and S.J. Chen. GM_Plan: a gate matrix layout algorithm based on artificial intelligence planning techniques. **IEEE Transactions on Computer-Aided Design**, v. 9, p. 836-845, 1990.
- Kashiwabara, T. and T. Fujisawa. NP-Completeness of the problem of finding a minimum clique number interval graph containing a given graph as a subgraph. In: Proceedings of the 1979 IEEE International Symposium on Circuits and Systems, Tokyo, Japan, 657-660, July 1979.
- Kececioğlu, J. and D. Sankoff. Exact and approximation algorithms for sorting by reversals, with application to genome rearrangement. **Algorithmica**, v. 13, p. 180-210, 1995.
- Kinnersley, N.G. The vertex separation number of a graph equals its path-width. **Information Processing Letters**, v. 42, p. 345-350, 1992.
- Kirkpatrick, S., Gelatt, D.C., and M. Vecchi. Optimization by simulated annealing. **Science**, v. 220, p. 671-680, 1983.
- Kirousis, L.M., and C.H. Papadimitriou, Interval graphs and searching. **Discrete Mathematics**, v. 55, p. 181-184, 1985.
- Kirousis, L.M. and C.H. Papadimitriou, Searching and pebbling. **Theoretical Computer Science**, v. 47, p. 205-218, 1986.
- Knuth, D., **The Art of Computer Programming. Vol. 3. Sorting and Searching.** Addison-Wesley, 1975.

- Kornai, A. and Z. Tuza, Narrowness, pathwidth, and their application in natural language processing. **Discrete Applied Mathematics**, v. 36, p. 87-92, 1992.
- Laarhoven, P.J.M., and Aarts, E.H.L., Simulated annealing: theory and practice. Kluwer Academic, Dordrecht, The Netherlands, 1987.
- Lengauer, T. Black and White pebbles and graph separation. **Acta Informatica**, v. 16, p. 465-475, 1981.
- Linhares, A. A glimpse at the metaphysics of Bongard problems. **Artificial Intelligence**, v. 121, n.1-2, p. 251-270, 2000.
- Linhares, A. and J.R.A. Torreão. Microcanonical optimization applied to the traveling salesman problem. **International Journal of Modern Physics C**, v. 9, p. 133-146, 1998.
- Linhares, A, and Yanasse, H.H. Connections between cutting-pattern sequencing, VLSI layouts, and flexible machines. Accepted for publication in **Computers & Operations Research**, 2001.
- Linhares, A., Yanasse, H.H. and J.R.A. Torreão. Linear gate assignment: a fast statistical mechanics approach. **IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems**, v.18, p. 1750-1758, 1999.
- Lopez, A.D. and H-F S. Law. A dense gate matrix layout method for MOS VLSI. **IEEE Transactions on Electronic Devices**, v. 27, p. 1671-1675, 1980.
- Madsen, O.B.G. An application of travelling salesman routines to solve pattern allocation problems in the glass industry. **Journal of the Operational Research Society**, v. 39, p. 249-256, 1988.
- Mak, K.T., and Morton, A.J. Distances between traveling salesman tours. **Discrete Applied Mathematics**, v. 58, p.281-291, 1995.
- Maniezzo, V., Mingozzi, A., Baldacci, R. A bionomic approach to the capacitated p-median problem. **Journal of Heuristics**, v. 4(3), pp. 263-280, 1998.

- Merz, P. and B. Freisleben. Fitness landscapes and memetic algorithm design. To appear in D. Corne, F. Glover, and M. Dorigo, **New Ideas in Optimization**, McGraw-Hill, 1999.
- Metropolis, N., Rosenbluth, A., Rosenbluth, M., Teller, A. and E. Teller. Equations of state calculations by fast computing machines. **Journal of Chemical Physics**, v. 21, p. 1087-1091, 1953.
- Möbius, A., Neklioudov, A., Díaz-Sánchez, A., Hoffmann, K. H., Fachat, A. and M. Schreiber. Optimization by thermal cycling. **Physical Review Letters**, v. 79, p. 4297-4301, 1997.
- Möhring, R.H., Graph problems related to gate matrix layout and PLA folding. **Computing**, v. 7, p. 17-51, 1990.
- Ohtsuki, T., Mori, H., Kuh, E.S., Kashiwabara, T. and T. Fujisawa, One-dimensional logic gate assignment and interval graphs. **IEEE Transactions on Circuits and Systems**, v. 26, 675-684, 1979.
- Osman, I.H., Kelly, J.P., **Meta-Heuristics : Theory & Applications**. Kluwer Academic Publishers, 1996.
- Ray, J.R. and R.W. Harris. Simulated annealing in the microcanonical ensemble. **Physical Review E**, v. 55, p. 5270-5274, 1997.
- Reeves, C.R. Landscapes, operators, and heuristic search. **Annals of Operations Research**, v. 86, p. 473-490, 1999.
- Shu, W., Wu, M.Y. and S.M. Kang. Improved net merging method for gate matrix layout. **IEEE Transaction on Computer Aided Design**, v. 7, p. 947-951, 1988.
- U. Singh, and C.Y. Roger Chen. From logic to symbolic layout for gate matrix. **IEEE Transactions on Computer Aided Design**, v. 11, p. 216-227, 1992.
- Tang, C.S. and E.V. Denardo. Models arising from a flexible manufacturing machine, part I: minimization of the number of tool switches. **Operations Research**, v. 36, 767-777, 1988.

- Torreão, J.R.A. and E. Roe. Microcanonical optimization applied to visual processing. **Physics Letters A**, v. 205, p. 377-382, 1995.
- Torreão, J.R.A. and J.L. Fernandes. Matching photometric stereo images. **Journal of the Optical Society of America A**, v. 15, p. 2966-2975, 1998.
- Tran, N. An easy case of sorting by reversals. **Journal of Computational Biology**, v.5, p. 741-746, 1998.
- Voss, S., Martello, S., Roucairol, C., Osman, I.H., **Meta-Heuristics : Advances and Trends in Local Search Paradigms for Optimization**. Kluwer Academic Publishers, 1998.
- Wing, O., Huang, S. and R. Wang. Gate matrix layout. **IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems**, v. 4, p. 220-231, 1985.
- Yanasse, H.H., Minimization of open orders – polynomial algorithms for some special cases. **Pesquisa Operacional**, v.16, p.1-26, 1996.
- Yanasse, H.H. On a pattern sequencing problem to minimize the maximum number of open stacks. **European Journal of Operational Research**, v.100, p. 454-463, 1997.
- Yanasse, H.H. A transformation for solving a pattern sequencing problem in the wood cut industry. **Pesquisa Operacional**, v.17, p.57-70, 1997a.
- Yanasse, H.H., Becceneri, J.C. and N.Y. Soma. Bounds for a problem of sequencing patterns. **Pesquisa Operacional**, v. 19, p.1-29, 1999.
- Yuen, B.J. Heuristics for sequencing cutting patterns. **European Journal of Operational Research**, v.55, p.183-190, 1991.
- Yuen, B.J. Improved heuristics for sequencing cutting patterns. **European Journal of Operational Research**, v.87, p. 57-64, 1995.
- Yuen, B.J., Richardson, K.V. Establishing the optimality of sequencing heuristics for cutting stock problems. **European Journal of Operational Research**, v. 84, p.590-598, 1995.